Université / École d'Ingénieur

Département Informatique / Génie Logiciel

Projet de Fin d'Études

Rapport de Stage

Conception et Implémentation d'un Pipeline de Prétraitement Intelligent pour la Base de Connaissances STM32Cube

---

Réalisé par :
Abir GHARBI

Encadrant entreprise :
[Nom du tuteur ST]

Encadrant académique :
[Nom du tuteur académique]

Entreprise d'accueil : STMicroelectronics
Division : AI & Digital Tools

Année universitaire : 2025–2026

Remerciements

Je tiens à exprimer ma sincère gratitude envers toute l'équipe STMicroelectronics pour leur accueil, leur encadrement et leur soutien tout au long de ce stage. Je remercie particulièrement [nom du tuteur] pour sa disponibilité et ses conseils avisés.

Table des matières

Remerciements **1**

Introduction Générale **8**

1 Présentation de l'Entreprise **10**

1.1 STMicroelectronics 10

1.1.1 Présentation Générale 10

1.1.2 Vision et Marchés 10

1.1.3 STMicroelectronics Tunis 11

1.2 Division Microcontrôleurs (MCD) 11

1.3 Équipe d'Accueil : Technical Marketing and Application Support 11

2 Contexte du Projet **13**

2.1 L'Écosystème STM32 et les Dépôts GitHub 13

2.1.1 Les Dépôts GitHub STMicroelectronics 14

2.2 Problématique 14

2.2.1 Critique de l'Existant 15

2.3 État de l'Art et Fondements Théoriques 16

2.3.1 Le Paradigme RAG 16

2.3.2 Preprocessing et Qualité de la KB 16

2.3.3 TF-IDF pour la Similarité 17

2.3.4 Alfred Vision pour la Description d'Images 17

2.4 Étude Comparative des Solutions Existantes 17

2.5 Synthèse de la Solution Proposée 18

2.6 Positionnement par Rapport à l'État de l'Art 19

2.7 Contributions Techniques Originales 19

2.8 Objectifs du Projet 20

2.9 Enjeux Éthiques et Responsabilité 20

2.10 Plan du Rapport 21

2.11 Les 25 Séries STM32 Couvertes 21

3 Spécification des Besoins et Méthodologie **23**

3.1 Identification des Acteurs . . . 23
3.2 Besoins Fonctionnels . . . 23
3.3 Besoins Non Fonctionnels . . . 24
3.4 Contraintes et Décisions de Conception . . . 25
3.5 Sélection de la Méthodologie : Scrum Adapté . . . 26
3.5.1 Organisation en Sprints . . . 26
3.5.2 Artefacts Scrum Utilisés . . . 27
3.6 Architecture Cible . . . 27

4 Pipeline de Data Preprocessing 28
4.1 Qu'est-ce qu'un Pipeline de Preprocessing ? . . . 28
4.1.1 Principes Fondamentaux . . . 28
4.1.2 Analogie avec l'ETL Classique . . . 28
4.2 Enjeux du Preprocessing pour le RAG . . . 29
4.2.1 Problème "Garbage In, Garbage Out" . . . 29
4.2.2 Exigences Spécifiques au Domaine Embedded . . . 29
4.3 Étapes Canoniques d'un Pipeline NLP . . . 30
4.3.1 1. Collecte (Data Ingestion) . . . 30
4.3.2 2. Nettoyage (Data Cleaning) . . . 30
4.3.3 3. Enrichissement (Feature Engineering) . . . 30
4.3.4 4. Mise en Relation (Linking & Similarity) . . . 31
4.3.5 5. Découpage (Chunking) . . . 31
4.3.6 6. Validation et Export . . . 31
4.4 Notre Pipeline : Mapping Théorie → Implémentation . . . 31
4.5 Métriques de Qualité d'un Pipeline . . . 32
4.6 Patterns d'Architecture Pipeline . . . 32
4.6.1 Pattern Batch vs Streaming . . . 32
4.6.2 Pattern "Rejouable par Stage" . . . 33
4.6.3 Pattern "Gate de Qualité" . . . 33

5 Méthodologie et Architecture 34
5.1 Approche Méthodologique . . . 34
5.2 Architecture Globale . . . 34
5.3 Choix Techniques . . . 34
5.4 Diagramme de Flux de Données . . . 35
5.5 Flux Source-to-Answer . . . 36
5.6 Architecture de Delivery et Upload KB . . . 37

6 Implémentation 38

6.1 Ingestion (Stage 1) . . . 38
6.1.1 Objectif . . . 38
6.1.2 Mécanisme . . . 38
6.1.3 Exemple de Sortie — Issue Brute . . . 38
6.2 Nettoyage (Stage 2) . . . 39
6.2.1 Nettoyage des Issues . . . 39
6.2.2 Exemple JSON — Issue Après Nettoyage . . . 40
6.2.3 Nettoyage des Fichiers . . . 40
6.2.4 Exemple JSON — Fichier Après Nettoyage . . . 40
6.3 Nettoyage V3 — Nettoyage Intelligent par Type (Stage 2b) . . . 41
6.3.1 Stratégies de Nettoyage par Type . . . 41
6.3.2 Filtrage CMSIS Device Headers . . . 43
6.3.3 Résultat du Filtrage — Exemple JSON . . . 43
6.3.4 Résultats du Nettoyage V3 . . . 44
6.3.5 Compatibilité et Non-Régression . . . 44
6.4 Enrichissement V2 (Stage 3) . . . 44
6.4.1 Heuristiques de Classification . . . 44
6.4.2 Exemple de Classification . . . 44
6.5 Similarité (Stage 4) . . . 45
6.5.1 Algorithme . . . 45
6.5.2 Exemple JSON — Issue avec Liens de Similarité . . . 45
6.6 Chunking V2 (Stage 5 — Optionnel) . . . 46
6.7 Delivery (Stage 7) . . . 47
6.7.1 Resolver Cases . . . 47
6.7.2 Exemple JSON — Resolver Case . . . 47
6.7.3 Diagnostic Cards . . . 48
6.7.4 Exemple JSON — Diagnostic Card . . . 48
6.7.5 Alfred Vision — Enrichissement d'Images . . . 50
6.8 Architecture Agents — Persona ST ChatGPT . . . 51
6.8.1 Vue d'ensemble . . . 52
6.8.2 Persona : ST GitHub Analyzer . . . 52
6.8.3 Tool 1 : KB Unstructured (Recherche Hybride) . . . 53
6.8.4 Tool 2 : NovaX Fallback . . . 53
6.8.5 Stratégie de Réponse (System Prompt) . . . 54
6.8.6 Pipeline comme Agent de Maintenance . . . 54

7 Configuration de la Plateforme ST ChatGPT 55
7.1 Architecture de la Plateforme . . . 55

7.2 Configuration du Tool KB . . . 55
7.2.1 Paramètres Retenus . . . 55
7.2.2 Justification : Recherche Hybride . . . 56
7.2.3 Justification : Seuil Sémantique à 0.55 . . . 56
7.2.4 Justification : 20 Documents Sémantiques + 12 Full-text . . . 58
7.2.5 Définition : Search Type (Hybrid) . . . 58
7.2.6 Tool Description . . . 59
7.3 Configuration du Tool NovaX (Fallback) . . . 60
7.3.1 Problème Identifié . . . 60
7.3.2 Fix : NovaX comme Fallback Strictement Secondaire . . . 60
7.3.3 Impact Mesuré . . . 60
7.4 System Prompt de la Persona . . . 61
7.5 Évolution Incrémentale du Quality Index . . . 61
7.5.1 Chronologie des Tests . . . 61
7.5.2 Analyse de l'Évolution . . . 63
7.5.3 Raisonnement de Chaque Amélioration . . . 64
7.6 Couverture Actuelle et Future . . . 65
7.6.1 13 Séries en Production . . . 65
7.6.2 4 Nouvelles Séries Exportées (Pipeline Terminé) . . . 66
7.6.3 8 Séries Configurées (Prêtes à Lancer) . . . 66

8 Résultats et Évaluation 67
8.1 Couverture des Données . . . 67
8.2 Impact du Nettoyage V3 sur la Qualité . . . 68
8.2.1 Comparaison V2 vs V3 — Réduction de Bruit . . . 68
8.2.2 Explication de l'Amélioration . . . 68
8.2.3 Datasources V3 en Production . . . 68
8.2.4 Filtre de Validité (is_valid) . . . 69
8.3 Métriques d'Évaluation par Stage . . . 69
8.3.1 Cadre Méthodologique et Références . . . 69
8.3.2 Échelle de Notation et Justification des Seuils . . . 70
8.3.3 Stage 1 : Ingestion — Complétude des Données Brutes . . . 70
8.3.4 Stage 2 : Cleaning — Qualité de Préparation Textuelle . . . 72
8.3.5 Stage 3 : Enrichissement — Classification par Heuristiques NLP . . . 72
8.3.6 Stage 3b : Enrichissement Alfred Vision — Images d'Issues . . . 74
8.3.7 Stage 3c : Enrichissement Figures PDF . . . 76
8.3.8 Synthèse de Confiance Alfred . . . 76
8.3.9 Stage 4 : Similarité TF-IDF . . . 77

8.3.10 Stage 5 : Delivery — Complétude des Artefacts Exportés 78
8.3.11 Résumé des Scores par Stage — Vue Synthétique 79
8.4 Score Global du Pipeline — Formule Recalculée 79
8.4.1 Formule de Scoring 79
8.4.2 Justification des Poids 80
8.4.3 Application Numérique (STM32CubeH7) 81
8.4.4 Interprétation et Comparaison 81
8.5 Étude Comparative : Alfred Générique vs Persona KB 82
8.5.1 Protocole Expérimental 82
8.5.2 Grille de Scoring Détaillée 83
8.5.3 Scorecard Comparative 83
8.5.4 Métriques Comparatives Synthétiques 84
8.5.5 Analyse par Catégorie de Questions 84
8.6 Contribution de Chaque Étape au Résultat Final 85
8.6.1 Chaîne de Valeur — Du Brut au QI 85
8.6.2 Impact Cumulatif sur le Quality Index 86
8.6.3 Matrice Valeur × Effort par Étape 87

9 Automatisation et CI/CD **88**
9.1 GitHub Actions — Mise à Jour Automatique 88
9.1.1 Architecture 2 Jobs 88
9.1.2 Déclenchement 89
9.1.3 Sécurité et Robustesse 89

10 Conclusion et Perspectives **90**
10.1 Bilan du Projet 90
10.2 Valeur Industrielle 91
10.3 Leçons Apprises 91
10.4 Perspectives 91

Références **93**

Annexes **96**

Table des figures

1.1 Logo officiel de STMicroelectronics 10

5.1 Vue d'ensemble du pipeline de preprocessing (7 stages) 35
5.2 Architecture détaillée du pipeline : stages, fichiers intermédiaires et branchement Alfred (draw.io) 36
5.3 Flux de données end-to-end : de la source GitHub à la réponse utilisateur . 37
5.4 Architecture de livraison et upload vers la Knowledge Base #793 37

6.1 Détail de l'étape d'ingestion : scripts et sorties (draw.io) 39
6.2 Arbre de décision du nettoyage V3 : routage par type de fichier avec taux de réduction (draw.io) 42
6.3 Détail du processus d'enrichissement et de calcul de similarité 46
6.4 Génération des Resolver Cases et Diagnostic Cards 49
6.5 Les 4 types d'artefacts delivery et leurs relations vers la KB #793 (draw.io) 50
6.6 Intégration Alfred Vision AI pour la description automatique d'images . . 51
6.7 Architecture multi-agents du système ST GitHub Analyzer (draw.io) . . . 52

7.1 Quality Index des stratégies Unstructured KB sur 96 questions — S1 maximise le rappel, S0 reste le point d'équilibre courant 58
7.2 Évolution du Quality Index au fil des améliorations incrémentales (draw.io) 63

8.1 Complétude d'ingestion par série — toutes métriques au-dessus du seuil EXCELLENT 71
8.2 Couverture d'enrichissement STM32CubeH7 — comparaison score vs seuil par champ 74
8.3 Décomposition du score global 87.1% — contribution pondérée de chaque stage, avec Alfred séparé par source visuelle 82
8.4 Comparaison Alfred vs Persona KB par catégorie de question 85
8.5 Comparaison des stratégies Unstructured KB sur 96 questions multi-séries 87

9.1 Architecture CI/CD : GitHub Actions + self-hosted runner 88

Introduction Générale

Le développement de systèmes embarqués basés sur les microcontrôleurs STM32 de STMicroelectronics est un domaine en constante évolution, où les ingénieurs font face quotidiennement à des problèmes techniques complexes : bugs de drivers, comportements inattendus des périphériques, incompatibilités entre versions firmware, configurations matérielles spécifiques. Pour résoudre ces problèmes, ils doivent aujourd'hui parcourir manuellement des milliers d'issues GitHub, des centaines de fichiers de documentation, et des dizaines de Release Notes répartis sur plus de 200 dépôts — un processus lent, fragmenté et inefficace.

Ce projet de fin d'études, réalisé au sein de l'équipe **Technical Marketing and Application Support** de la division Microcontrôleurs (MCD) du site STMicroelectronics de Tunis, propose une solution à cette problématique : la conception et l'implémentation d'un **pipeline de prétraitement intelligent** qui transforme automatiquement les données brutes des dépôts GitHub STM32Cube en une base de connaissances structurée. Cette base alimente un chatbot IA spécialisé ("ST GitHub Analyzer") déployé sur la plateforme interne ST ChatGPT, capable de fournir des diagnostics précis, sourcés et actionnables aux ingénieurs.

Le pipeline couvre les **25 séries** de la famille STM32 (13 en production, 12 configurées) et produit 4 types d'artefacts complémentaires : issues structurées, fichiers documentés, resolver cases (chaînes issue→PR→commit) et diagnostic cards (analyses de pannes structurées). Un enrichissement IA multi-modal par Alfred Vision rend le contenu visuel (captures d'écran, figures PDF) exploitable par la recherche.

Les résultats démontrent une **amélioration nette de la pertinence** par rapport à un LLM générique sur les questions techniques STM32, avec un Quality Index atteignant 94/100 sur la série pilote H7 grâce aux Diagnostic Cards.

Ce rapport s'organise comme suit :

— **Chapitre 1** — Présentation de l'entreprise : ST, le site de Tunis, l'équipe d'accueil

— **Chapitre 2** — Contexte du projet : problématique, critique de l'existant, solution proposée

— **Chapitre 3** — Spécification des besoins : acteurs, besoins fonctionnels/non fonctionnels, méthodologie Scrum

— **Chapitre 4** — Architecture et conception : architecture cible, choix techniques, flux de données

— **Chapitre 5** — Implémentation : les 7 stages du pipeline, enrichissement Alfred, delivery

— **Chapitre 6** — Tests et évaluation : métriques par stage, Quality Index, étude comparative

— **Chapitre 7** — Configuration de la plateforme ST ChatGPT et automatisation CI/CD

— **Chapitre 8** — Conclusion et perspectives

Chapitre 1

Présentation de l'Entreprise

1.1 STMicroelectronics

1.1.1 Présentation Générale

STMicroelectronics est une entreprise mondiale de semi-conducteurs dédiée à l'amélioration de la vie quotidienne grâce à des solutions électroniques innovantes [4]. Avec l'un des portefeuilles de produits les plus étendus de l'industrie, ST dessert plus de 200 000 clients dans le monde, réalisant un chiffre d'affaires net de 13,27 milliards USD en 2024 [5].

L'entreprise a été fondée en juin 1987 sous le nom de **SGS-THOMSON Microelectronics**, suite à la fusion de SGS Microelettronica (Italie) et de Thomson Semiconductors (France). Le nom "STMicroelectronics" a été officiellement adopté en mai 1998. Avec plus de 80 bureaux commerciaux et 14 sites de fabrication principaux à travers le monde, l'entreprise emploie plus de 50 000 personnes [5].

[Logo STMicroelectronics — exporter depuis draw.io]

FIGURE 1.1 – Logo officiel de STMicroelectronics

1.1.2 Vision et Marchés

STMicroelectronics a fait de l'innovation technologique une priorité stratégique depuis plus de 25 ans. L'entreprise réalise des investissements orientés marché dans le développement technologique, avec l'objectif de commercialiser des innovations de pointe créant une valeur immédiate pour ses clients. Aujourd'hui, ST livre des produits toujours plus petits, plus rapides, plus économes en énergie et dotés de nouvelles fonctionnalités.

Portée par une culture mondiale de *lean manufacturing*, ST adresse **trois marchés principaux** :

— **Automobile** (37%) : capteurs, microcontrôleurs, gestion de puissance pour véhicules électriques et autonomes

— **Industriel** (30%) : automatisation, contrôle moteur, IoT industriel, smart grid

— **Électronique personnelle** (20%) : smartphones, wearables, audio, gaming

— **Communications et équipements** (13%) : infrastructure réseau, serveurs

ST propose des modèles d'engagement allant des systèmes embarqués spécifiques à l'application jusqu'aux produits standard spécialisés (ASSP).

1.1.3 STMicroelectronics Tunis

Le site **ST Tunis**, situé dans le **Technopark El Ghazela**, a débuté ses opérations en 2001 avec neuf ingénieurs. Aujourd'hui, il compte plus de **200 employés** travaillant dans les domaines du design, des outils, des applications, de la qualité et du support [5]. Le site participe à chaque étape, depuis la conception initiale de nouveaux circuits jusqu'à la vérification finale de leur fonctionnalité avant la production de masse.

1.2 Division Microcontrôleurs (MCD)

La **Microcontroller Division (MCD)** est dédiée à la conception de microcontrôleurs, de plateformes de démonstration et de solutions pour les applications grand public. Elle est responsable du développement de drivers et de logiciels embarqués — en particulier la validation et le développement des bibliothèques et firmware qui pilotent les microcontrôleurs ST.

1.3 Équipe d'Accueil : Technical Marketing and Application Support

Ce projet a été réalisé au sein de l'équipe **Technical Marketing and Application Support** rattachée à la MCD sur le site de Tunis. Cette équipe comprend deux sous-équipes :

— **L'équipe GitHub** : assure le support client via les dépôts GitHub publics de ST-Microelectronics, publie les mises à jour des drivers HAL, et maintient les projets d'exemples pour faciliter le développement d'applications basées sur les microcontrôleurs STM32.

— **L'équipe Maintenance et Validation** : garantit la satisfaction client par la maintenance des bugs et la validation fonctionnelle des microcontrôleurs fabriqués et de

leurs versions HAL associées. Cette équipe représente la jonction entre les processus de conception logicielle/matérielle et l'industrialisation.

C'est dans ce contexte que le besoin d'un **chatbot IA spécialisé STM32** a émergé : face au volume croissant d'issues GitHub (plusieurs milliers par an, répartis sur 200+ dépôts) et à la richesse inexploitée de ces données, l'équipe a identifié l'opportunité d'automatiser le support de premier niveau par un pipeline intelligent de prétraitement alimentant une base de connaissances structurée.

Chapitre 2

Contexte du Projet

2.1 L'Écosystème STM32 et les Dépôts GitHub

Au cœur de l'offre de ST se trouve la famille **STM32**, une gamme de microcontrôleurs basés sur les cœurs ARM Cortex-M (M0, M0+, M3, M4, M7, M33, M55) et Cortex-A. Lancée en 2007, la famille STM32 compte aujourd'hui plus de **1 500 références** regroupées en plus de 30 séries, chacune ciblant un profil d'application spécifique :

Série	**Cœur**	**Cible applicative**
STM32F0/F1/F2	Cortex-M0/M3	Entrée de gamme, migration depuis 8-bit
STM32F3/F4	Cortex-M4F	Applications signal mixte, contrôle moteur, audio
STM32F7	Cortex-M7	Haute performance, graphisme, connectivité
STM32G0/G4	Cortex-M0+/M4	Nouvelle génération mainstream, efficacité énergétique
STM32H5/H7/H7RS	Cortex-M33/M7	Ultra-haute performance, dual-core, temps réel
STM32L0/L1/L4/L5	Cortex-M0+/M3/M4/M33	Ultra-basse consommation, IoT, wearables
STM32U0/U3/U5	Cortex-M0+/M33	Ultra-basse consommation nouvelle génération
STM32WB/WBA/WL	Cortex-M4+M0/M33	Connectivité sans fil (BLE, Thread, LoRa, Sub-GHz)
STM32N6	Cortex-M55+NPU	IA embarquée, vision par ordinateur

Pour chaque série, ST fournit un **Firmware Package** complet sous le nom **STM32Cube**,

comprenant :

- Le **HAL** (Hardware Abstraction Layer) : couche d'abstraction portable entre séries
- Les **drivers LL** (Low-Layer) : accès registre optimisé pour la performance
- Les **BSP** (Board Support Packages) : drivers spécifiques aux cartes de développement (NUCLEO, Discovery, Eval)
- Les **CMSIS Device Headers** : définitions de registres ARM standardisées
- Des **Middlewares** : FreeRTOS, LwIP, FatFs, USB, TouchGFX, LoRaWAN
- Des **exemples d'application** : projets prêts à compiler par board et par IDE

2.1.1 Les Dépôts GitHub STMicroelectronics

Depuis 2019, STMicroelectronics publie l'intégralité de ses firmware packages STM32Cube en open source sur GitHub (`github.com/STMicroelectronics`). Cette présence GitHub représente un écosystème considérable :

- **25 dépôts principaux** (un par série MCU : `STM32CubeH7`, `STM32CubeF4`, etc.)
- **200+ sous-dépôts** pour les drivers individuels (`stm32h7xx-hal-driver`, `stm32f4xx-nucleo-bsp`, `cmsis_device_h7`, etc.)
- **Des milliers d'issues** ouvertes par la communauté (bug reports, questions d'utilisation, demandes de fonctionnalités)
- **Des centaines de Pull Requests** avec des corrections de code et des discussions techniques
- De la **documentation technique** embarquée : Release Notes (HTML), README (Markdown), documentation PDF

Ce projet porte sur les **25 séries** de la famille STM32 : **13 séries en production** (C0, F0, F1, F2, F3, F4, F7, G0, H5, H7, H7RS, U5, WL) couvrant 47 dépôts et **12 séries supplémentaires** configurées et prêtes au déploiement (G4, L0, L1, L4, L5, N6, U0, U3, WB, WB0, WBA, WL3). La couverture totale cible est de **25 séries MCU** avec plus de 200 sous-dépôts associés.

2.2 Problématique

Les ingénieurs STM32 — qu'ils soient internes à ST ou clients — rencontrent quotidiennement des problèmes techniques lors du développement firmware : bugs HAL, comportements inattendus des périphériques, incompatibilités entre versions, configurations complexes. Pour résoudre ces problèmes, ils doivent aujourd'hui parcourir manuellement les ressources disponibles, un processus qui souffre de quatre limitations majeures :

1. **Volume et dispersion** : les données sont réparties sur plus de 200 dépôts GitHub, totalisant des milliers d'issues, des centaines de PRs et des dizaines de milliers de fichiers source. Trouver l'information pertinente revient à chercher une aiguille dans une botte de foin numérique.
2. **Bruit et hétérogénéité** : le contenu des issues mélange du texte technique utile avec du bruit considérable — templates vides, messages automatiques de bots GitHub, discussions hors-sujet, doublons, markdown malformé. Les fichiers source contiennent des licences volumineuses (30% du texte pour certains fichiers) et les headers CMSIS dépassent 1,7 Mo de définitions bit-à-bit inutiles pour le diagnostic.
3. **Fragmentation des connaissances** : la résolution d'un bug implique souvent de recouper une issue, une PR avec sa discussion, un commit dans un sous-dépôt différent, et une Release Note qui documente le correctif. Ces liens ne sont pas toujours explicites dans GitHub et nécessitent une investigation manuelle fastidieuse.
4. **Opacité du contenu visuel** : de nombreuses issues contiennent des captures d'écran (erreurs IDE STM32CubeIDE, oscillogrammes, configurations CubeMX, schémas de câblage) qui portent une information diagnostique critique mais sont totalement invisibles pour la recherche textuelle. De même, les Release Notes PDF contiennent des figures techniques non indexables.

La question centrale de ce projet est donc : *comment transformer cette masse de données brutes, hétérogènes et fragmentées en une base de connaissances structurée, exploitable par un chatbot IA capable de fournir des diagnostics précis, sourcés et actionnables aux ingénieurs STM32 ?*

2.2.1 Critique de l'Existant

Avant ce projet, les solutions disponibles présentaient des limitations significatives :

Solution existante	Limitation principale	Impact utilisateur
Recherche GitHub native	Recherche textuelle brute, pas de filtrage par composant, layer ou sévérité	L'ingénieur parcourt 50+ issues sans garantie de trouver la bonne
Alfred (LLM ST)	Connaissances génériques, aucun accès aux issues GitHub spécifiques	Conseils généraux sans root cause précise ni traçabilité

Stack Overflow	Communauté large mais non spécifique ST, pas de traçabilité vers les correctifs officiels	Solutions non vérifiées, souvent obsolètes ou inapplicables
Documentation PDF	Non indexable, figures opaques, pas de lien avec les bugs connus	L'ingénieur ne peut pas corréler son problème avec les bugs documentés
Support direct (email/forum)	Temps de réponse de 48h à 2 semaines	Blocage du développement client

Constat quantifié : Sur un benchmark de 20 questions techniques couvrant 5 séries MCU, Alfred générique obtient un score de 116/200 (58%) tandis que notre solution avec Knowledge Base obtient 161/200 (80,5%), soit un **gain de +39%** en pertinence, actionnabilité et traçabilité (cf. Section **??** pour le détail de l'étude).

2.3 État de l'Art et Fondements Théoriques

2.3.1 Le Paradigme RAG

Notre projet repose sur le paradigme **RAG** (Retrieval-Augmented Generation) [1], qui combine la recherche documentaire avec la génération de texte par un LLM. Concrètement dans notre système, lorsqu'un ingénieur pose une question, le moteur de recherche de la KB #793 récupère les documents les plus pertinents (issues, fichiers, diagnostic cards), puis le LLM (GPT-5.1 via Azure OpenAI) génère une réponse ancrée dans ces documents avec des citations vérifiables.

Ce paradigme nous apporte trois avantages concrets :

— **Réduction des hallucinations** : les réponses sont ancrées dans des issues GitHub réelles, pas dans les connaissances génériques du modèle

— **Mise à jour sans ré-entraînement** : quand de nouvelles issues apparaissent, il suffit de relancer le pipeline et uploader — pas besoin de ré-entraîner le modèle

— **Traçabilité** : chaque réponse cite un numéro d'issue, une URL GitHub, une version firmware

2.3.2 Preprocessing et Qualité de la KB

La qualité des réponses RAG dépend directement de la qualité des documents indexés — c'est le principe "**Garbage In, Garbage Out**". Dans notre projet, nous avons constaté expérimentalement que :

— Les headers CMSIS bruts (1,7 Mo de `#define`) polluent le contexte du LLM avec du bruit inutile → d'où le filtre V3 (−96%)

— Les balises HTML des Release Notes diluent le signal technique → d'où le nettoyage `strip_html()` (−33%)

— Les licences volumineuses dans le code source occupent de l'espace dans les chunks → d'où la suppression automatique

Pour le découpage en chunks, nous utilisons une approche **paragraph-based** pour les issues et **section-based** pour les fichiers, avec une taille maximale de 512 tokens et un overlap de 50 tokens.

2.3.3 TF-IDF pour la Similarité

Pour le calcul de similarité inter-issues, nous utilisons **TF-IDF** (Term Frequency – Inverse Document Frequency) [2], qui pondère les termes par leur spécificité dans le corpus. Nous avons choisi TF-IDF plutôt que des embeddings neuronaux car il est déterministe, interprétable et ne nécessite pas de modèle pré-entraîné — ce qui est cohérent avec notre choix d'heuristiques plutôt que de ML pour la classification.

2.3.4 Alfred Vision pour la Description d'Images

Pour rendre les captures d'écran et figures PDF "cherchables" par le RAG, nous utilisons **Alfred Vision**, le service de description d'images interne de ST basé sur Azure OpenAI. Alfred reçoit une image et un prompt spécialisé STM32, et produit une description textuelle orientée diagnostic (noms de périphériques, messages d'erreur, valeurs de registres).

2.4 Étude Comparative des Solutions Existantes

Avant de concevoir notre pipeline, nous avons évalué les solutions existantes pour le support technique assisté par IA :

Solution	**Ce qu'elle fait bien**	**Pourquoi insuffisante pour nous**
Recherche GitHub native	Gratuite, intégrée	Pas de sémantique, pas de filtrage par composant/layer, résultats noyés dans le bruit

Alfred (LLM ST générique)	Toujours disponible, connaissances larges	Pas d'accès aux issues GitHub, conseils génériques sans root cause (score 116/200)
Stack Overflow	Communauté large	Non spécifique ST, solutions non vérifiées et souvent obsolètes
GitHub Copilot	Complétion de code intelligente	Pas de KB custom, aucun accès aux issues et bugs connus
Notre approche	Pipeline domain-specific, 4 artefacts, enrichissement IA	Pipeline custom à maintenir

Conclusion : aucune solution existante ne combine le preprocessing domain-aware, les artefacts de diagnostic structurés et l'enrichissement d'images. C'est ce constat qui a motivé la conception de notre pipeline.

2.5 Synthèse de la Solution Proposée

Face à cette problématique et aux limites des solutions existantes, nous proposons un **pipeline intelligent de prétraitement en 7 étapes** qui transforme les données GitHub brutes en une Knowledge Base structurée, alimentant un chatbot IA spécialisé STM32 déployé sur la plateforme ST ChatGPT. L'architecture repose sur trois piliers fondamentaux :

1. **Pipeline de preprocessing modulaire** : Ingestion → Nettoyage (V1+V3 par type) → Enrichissement NLP → Similarité TF-IDF → Chunking → Validation schéma → Delivery. Chaque étape est **rejouable indépendamment**, produit des artefacts JSON traçables, et peut être évaluée en isolation.
2. **4 types d'artefacts complémentaires** couvrant l'intégralité du spectre de connaissances :
   - **Issues structurées** : symptôme, root cause, fix, métadonnées enrichies (layer, component, severity, board)
   - **Fichiers documentés** : HAL/BSP/CMSIS/PDF extraits et nettoyés par type avec filtrage intelligent
   - **Resolver Cases** : chaînes Issue → PR → Commit avec preuves de résolution et fichiers modifiés

   - **Diagnostic Cards** : analyses structurées (symptôme → hypothèses → validation → fix → prévention)
3. **Enrichissement IA multi-modal** : Alfred Vision AI convertit automatiquement les captures d'écran des issues et les figures techniques des PDFs en descriptions textuelles cherchables, rendant le contenu visuel exploitable par la recherche RAG.

2.6 Positionnement par Rapport à l'État de l'Art

Notre approche se positionne comme une **solution RAG verticale** (domain-specific) par opposition aux solutions RAG horizontales (génériques). Les contributions originales par rapport à l'état de l'art sont :

1. **Preprocessing domain-aware** : là où les solutions génériques indexent le contenu brut, notre pipeline applique un nettoyage adapté au type de fichier (HTML pour les Release Notes, filtrage intelligent pour les CMSIS headers, suppression de licence pour le code source)
2. **Enrichissement métadonnées par heuristiques** : classification automatique des issues par layer architecturale, composant matériel, board, sévérité et type — sans données d'entraînement supervisées, grâce à des heuristiques regex spécifiques au domaine STM32
3. **Artefacts de diagnostic structurés** : les Diagnostic Cards et les Resolver Cases sont des formats d'artefacts originaux qui n'existent pas dans les pipelines RAG standards. Ils ont démontré un impact majeur sur la qualité (+33 points de Quality Index)
4. **Enrichissement Vision IA des images** : l'utilisation d'Alfred Vision pour rendre cherchables les captures d'écran et les figures PDF est une contribution technique inédite dans le contexte STM32

2.7 Contributions Techniques Originales

1. **Pipeline modulaire 7 stages** rejouable indépendamment par étape avec artefacts JSON traçables
2. **Nettoyage V3 par type** : routage intelligent avec réduction de 96% sur les headers CMSIS et 33% sur le HTML
3. **Filtre CMSIS intelligent** ne conservant que les sections à valeur diagnostique (IRQ, base addresses, TypeDef)
4. **4 types d'artefacts delivery** complémentaires : issues, files, resolver cases, diagnostic cards

5. **Heuristiques NLP domain-specific** pour la classification STM32 (layer, component, severity, board, issue_kind)
6. **Description automatique d'images** par Alfred Vision AI (captures d'écran issues + figures PDF)
7. **Architecture KB-first + NovaX fallback** avec description restrictive évitant le court-circuit de la KB
8. **Workflow Delete+Add** résolvant les conflits de noms lors de l'upload des data-sources
9. **Couverture multi-séries** : 13 séries en production, 12 configurées, processus reproductible en 3 étapes

2.8 Objectifs du Projet

Les objectifs de ce projet de fin d'études sont les suivants :

1. **Concevoir** un pipeline de prétraitement automatisé capable d'ingérer les données brutes depuis l'API GitHub, de les nettoyer, enrichir et structurer pour un système RAG
2. **Implémenter** les 7 étapes du pipeline (ingestion, nettoyage V1+V3, enrichissement NLP, similarité TF-IDF, chunking, validation, delivery) avec une architecture modulaire et rejouable
3. **Déployer** un chatbot IA spécialisé STM32 sur la plateforme ST ChatGPT, alimenté par la Knowledge Base générée par le pipeline
4. **Évaluer** la qualité du système de bout en bout via un Quality Index sur 52 questions et une étude comparative avec Alfred générique
5. **Automatiser** la mise à jour continue via CI/CD (GitHub Actions) et des scripts d'orchestration
6. **Industrialiser** la solution pour couvrir l'intégralité des 25 séries STM32

2.9 Enjeux Éthiques et Responsabilité

L'utilisation de l'intelligence artificielle dans un contexte de support technique industriel soulève plusieurs enjeux éthiques que nous avons pris en compte :

1. **Traçabilité et transparence** : chaque réponse du chatbot est accompagnée de citations vérifiables (numéro d'issue, URL GitHub, version firmware). L'utilisateur peut toujours remonter à la source pour vérifier l'exactitude de l'information. Cette traçabilité est un choix de conception délibéré.

2. **Fiabilité et hallucinations** : le paradigme RAG réduit significativement les hallucinations par rapport à un LLM générique, car les réponses sont ancrées dans des documents factuels. Néanmoins, le risque n'est pas nul — d'où l'importance des gates de validation et du Quality Index régulier.
3. **Confidentialité des données** : les données sources proviennent des dépôts GitHub de STMicroelectronics (issues, fichiers, PRs). Bien que ces dépôts soient accessibles sur GitHub, les données traitées, la Knowledge Base générée et la plateforme ST ChatGPT sont des ressources **internes à ST**. Le pipeline ne traite aucune donnée personnelle des utilisateurs GitHub au-delà des identifiants publics (pseudo, commentaires publics).
4. **Biais de couverture** : certaines séries MCU ont plus d'issues que d'autres, ce qui crée un biais de couverture. Les séries historiques (F4, H7) sont mieux représentées que les séries récentes (N6, U0). Nous documentons cette limitation dans le chapitre d'évaluation.
5. **Impact sur l'emploi** : le chatbot ne remplace pas les ingénieurs de support mais **augmente leur productivité** en automatisant la recherche de premier niveau. Les cas complexes nécessitent toujours une expertise humaine.

2.10 Plan du Rapport

Le plan détaillé est présenté dans l'Introduction Générale (page précédente).

2.11 Les 25 Séries STM32 Couvertes

Le pipeline est conçu pour couvrir l'intégralité de l'écosystème STM32. Le tableau suivant détaille l'état de chaque série :

Série	**Statut**	**Repos principaux**	**Issues**	**Sous-dépôts drivers**
STM32CubeC0	Production	1	∼50	3 (HAL, CMSIS, BSP)
STM32CubeF0	Production	1	∼120	3
STM32CubeF1	Production	1	∼180	3
STM32CubeF2	Production	1	∼60	3
STM32CubeF3	Production	1	∼100	3
STM32CubeF4	Production	1	∼800	16 (HAL, CMSIS, 14 BSP)

STM32CubeF7	Production	1	∼350	6
STM32CubeG0	Production	1	∼150	3
STM32CubeH5	Production	1	∼200	6
STM32CubeH7	Production	1	∼500	12 (HAL, CMSIS, 10 BSP)
STM32CubeH7RS	Production	1	∼80	3
STM32CubeU5	Production	1	∼350	8
STM32CubeWL	Production	1	∼150	5
STM32CubeG4	Configuré	1	∼49	4 (HAL, CMSIS, 2 BSP)
STM32CubeL0	Configuré	1	∼31	4
STM32CubeL1	Configuré	1	∼13	4
STM32CubeL4	Configuré	1	∼94	4
STM32CubeL5	Configuré	1	—	4
STM32CubeN6	Configuré	1	—	3
STM32CubeU0	Configuré	1	—	3
STM32CubeU3	Configuré	1	—	3
STM32CubeWB	Configuré	1	—	3
STM32CubeWB0	Configuré	1	—	3
STM32CubeWBA	Configuré	1	—	3
STM32CubeWL3	Configuré	1	—	3
Total		**25**	**∼3 350**	**∼110**

Chapitre 3

Spécification des Besoins et Méthodologie

3.1 Identification des Acteurs

Le système implique plusieurs catégories d'acteurs aux rôles et attentes distincts :

Acteur	Rôle	Interaction avec le système
Ingénieur STM32 (client/interne)	Utilisateur final du chatbot	Pose des questions techniques, reçoit des diagnostics sourcés avec liens GitHub
Équipe Support ST	Mainteneur de la KB	Lance le pipeline, valide les exports, surveille le Quality Index
Administrateur KB	Gestionnaire de la plateforme	Crée les datasources, configure les personas et tools sur ST ChatGPT
Pipeline automatisé	Agent de maintenance	Ingère périodiquement les nouvelles issues, met à jour la KB, déclenche les validations
GitHub API	Source de données	Fournit les issues, PRs, commits, fichiers via l'API REST
Alfred Vision AI	Service d'enrichissement	Décrit automatiquement les images et figures PDF

3.2 Besoins Fonctionnels

ID	Besoin	Priorité	Critère d'acceptation
BF1	Ingérer automatiquement les issues, PRs, commits et fichiers depuis GitHub	Critique	100% des issues récupérées pour un repo donné
BF2	Nettoyer le contenu textuel en supprimant le bruit (bots, HTML, licences)	Critique	Ratio signal/bruit $> 90\%$
BF3	Enrichir les issues avec des métadonnées structurées (layer, component, board, severity)	Haute	Couverture layer $\geq 85\%$, component $\geq 70\%$
BF4	Calculer la similarité inter-issues et établir des liens	Moyenne	Liens pertinents (précision $> 80\%$)
BF5	Générer 4 types d'artefacts delivery (issues, files, resolver cases, diagnostic cards)	Critique	Conformité 100% au JSON Schema
BF6	Uploader les artefacts vers la KB #793 sur ST ChatGPT	Critique	Datasources créées et indexées
BF7	Décrire automatiquement les images des issues par Alfred Vision	Haute	Taux de succès $\geq 95\%$
BF8	Décrire automatiquement les figures des PDFs de Release Notes	Moyenne	Intégration dans le texte livré
BF9	Valider les artefacts contre un JSON Schema avant upload	Critique	Gate bloquant en cas d'erreur
BF10	Fournir un chatbot IA qui répond aux questions STM32 avec citations	Critique	Quality Index $\geq 80/100$

3.3 Besoins Non Fonctionnels

ID	Besoin	Catégorie	Mesure

BNF1	Le pipeline doit être rejouable par étape sans effet de bord	Modularité	Chaque stage indépendant avec entrée/sortie JSON
BNF2	Le pipeline doit traiter une série complète en moins de 30 minutes	Performance	Temps mesuré sur H7 (12 repos, 500 issues)
BNF3	Le pipeline doit être reproductible (mêmes entrées → mêmes sorties)	Idempotence	Tests de non-régression
BNF4	Le code doit être maintenable et extensible à de nouvelles séries	Maintenabilité	Ajout d'une série en 3 étapes documentées
BNF5	Les données sources proviennent des dépôts GitHub ST ; la KB et la plateforme restent internes	Sécurité / Confidentialité	Aucune donnée personnelle traitée au-delà des identifiants publics
BNF6	Le système doit supporter 25+ séries MCU sans refactoring	Scalabilité	Configs séparées par série
BNF7	La mise à jour de la KB doit être automatisable	Automatisation	Workflow CI/CD fonctionnel
BNF8	Le chatbot doit répondre en moins de 15 secondes	Temps de réponse	Mesure via API ST ChatGPT

3.4 Contraintes et Décisions de Conception

Le projet est soumis à plusieurs contraintes qui ont orienté les choix techniques :

Contrainte	**Impact**	**Décision prise**
Rate limiting GitHub API (5000 req/h)	Limite la vitesse d'ingestion	Sleep adaptatif + retry exponentiel

Plateforme ST ChatGPT (Azure OpenAI)	Format et API imposés	Export JSON avec processor params base64
Pas de données d'entraînement supervisées	Impossible d'utiliser du ML classique pour la classification	Heuristiques regex domain-specific
Headers CMSIS > 1.7 Mo	Explosion de la taille des chunks et bruit dans le retrieval	Filtre V3 intelligent (−96%)
Images opaques dans les issues	Contenu diagnostique invisible pour la recherche textuelle	Enrichissement Alfred Vision AI
Multi-séries (25+ repos)	Risque de dilution du contexte dans la KB	Datasources séparées par série + métadonnées de filtrage

3.5 Sélection de la Méthodologie : Scrum Adapté

Le projet a été conduit selon une méthodologie **Scrum adaptée** au contexte d'un projet de fin d'études, avec des sprints de 2 semaines et des revues régulières avec l'encadrant entreprise.

3.5.1 Organisation en Sprints

Sprint	**Période**	**Objectifs et livrables**
Sprint 1	Fév – Mars 2026	Étude de l'existant, état de l'art RAG/NLP, POC ingestion H7
Sprint 2	Mars 2026	Pipeline H7 complet (ingestion → delivery), première version KB
Sprint 3	Avr 2026	Création persona ST ChatGPT, calibration recherche hybride, premiers Quality Index
Sprint 4	Avr – Mai 2026	Diagnostic Cards + Resolver Cases, nettoyage V3, QI = 94 sur H7
Sprint 5	Mai 2026	Extension multi-séries (F4, H5, U5, WL), étude comparative Alfred vs Persona
Sprint 6	Mai – Juin 2026	Alfred Vision (images issues + PDF), enrichissement 13 séries

Sprint 7	Juin 2026	Nouvelles séries (G4, L0, L1, L4), automatisation CI/CD, évaluation finale

3.5.2 Artefacts Scrum Utilisés

— **Product Backlog** : liste priorisée des fonctionnalités (pipeline stages, types d'artefacts, enrichissement IA)

— **Sprint Backlog** : sélection des tâches pour chaque sprint de 2 semaines

— **Daily Stand-up** : point quotidien avec l'équipe ST (15 min)

— **Sprint Review** : démonstration des livrables à l'encadrant (Quality Index, démo chatbot)

— **Sprint Retrospective** : analyse des blocages et améliorations (ex : API rate limiting, JSON corruption)

3.6 Architecture Cible

L'architecture cible du système s'articule en trois couches :

1. **Couche Data (Sources)** : les dépôts GitHub STMicroelectronics fournissent les données brutes via l'API REST. Les fichiers source sont récupérés par clonage local des repositories.
2. **Couche Processing (Pipeline)** : le pipeline Python en 7 étapes transforme les données brutes en artefacts structurés JSON. Chaque étape lit un fichier d'entrée et produit un fichier de sortie, permettant la rejouabilité et le débogage en isolation.
3. **Couche Delivery (Plateforme)** : les artefacts JSON sont uploadés vers la KB #793 sur la plateforme ST ChatGPT (Azure OpenAI). La persona "ST GitHub Analyzer" interroge cette KB via une recherche hybride (semantic + full-text) et génère des réponses sourcées.

Note : le diagramme d'architecture détaillé est présenté en Figure 5.2 au Chapitre 5, et les diagrammes draw.io exportés sont référencés en Annexes.

Chapitre 4

Pipeline de Data Preprocessing

4.1 Qu'est-ce qu'un Pipeline de Preprocessing ?

Un **pipeline de preprocessing** (ou pipeline de prétraitement de données) est une chaîne de transformations automatisées qui convertit des données brutes et non structurées en données propres, enrichies et exploitables par un système en aval (ici, un moteur RAG).

4.1.1 Principes Fondamentaux

1. **Séquentialité** : Chaque étape prend en entrée la sortie de l'étape précédente
2. **Idempotence** : Rejouer une étape avec les mêmes entrées produit les mêmes sorties
3. **Modularité** : Chaque étape est indépendante et remplaçable
4. **Traçabilité** : Chaque transformation est documentée et versionnable
5. **Validation** : Des gates de qualité empêchent la propagation d'erreurs

4.1.2 Analogie avec l'ETL Classique

Le pipeline de preprocessing s'apparente à un processus **ETL** (Extract, Transform, Load) adapté au domaine NLP/RAG :

Phase ETL	**Équivalent Pipeline**	**Description**
Extract	Ingestion	Extraction depuis l'API GitHub (issues, fichiers, PRs)
Transform	Cleaning + Enrichment + Chunking	Normalisation, classification NLP, découpage
Load	Delivery + Upload	Export JSON + upload vers la KB cloud

4.2 Enjeux du Preprocessing pour le RAG

Le paradigme RAG (Retrieval-Augmented Generation) est extrêmement sensible à la qualité des documents indexés. Un mauvais preprocessing cause directement :

4.2.1 Problème "Garbage In, Garbage Out"

— **Bruit** : du texte non pertinent (signatures email, messages bot, headers GitHub) dilue les signaux techniques utiles

— **Incomplétude** : un document sans métadonnées (layer, component) ne peut pas être filtré efficacement lors de la recherche

— **Duplication** : des contenus dupliqués saturent le contexte du LLM avec des informations redondantes

— **Opacité** : les images (captures d'écran, logs visuels) sont invisibles pour la recherche textuelle

4.2.2 Exigences Spécifiques au Domaine Embedded

Le domaine STM32 pose des défis supplémentaires par rapport au preprocessing de texte générique :

Défi	**Impact sur le Preprocessing**
Vocabulaire technique dense	Les noms d'API (`HAL_SPI_Transmit_DMA`), les registres (`CR1`), et les codes d'erreur doivent être préservés intacts
Contexte multi-couches	Un problème peut impliquer simultanément le hardware, le HAL, le middleware et l'application — le pipeline doit identifier la couche concernée
Dépendance à la version	Un workaround valide pour CubeH7 v1.9 peut être obsolète en v1.11
Images techniques	Oscillogrammes, screenshots IDE, schémas de connexion contiennent des informations critiques
Multi-repository	Une série MCU (ex : H7) s'étend sur 12+ repos (HAL, BSP, CMSIS, exemples)

4.3 Étapes Canoniques d'un Pipeline NLP

La littérature identifie les étapes suivantes pour un pipeline de preprocessing NLP appliqué au RAG [1] :

4.3.1 1. Collecte (Data Ingestion)

Récupération exhaustive des données depuis les sources :

— API REST (GitHub, JIRA, Confluence)

— Systèmes de fichiers (repositories clonés)

— Bases de données internes

Critère de qualité : complétude (aucune donnée manquée) et fraîcheur (données récentes).

4.3.2 2. Nettoyage (Data Cleaning)

Suppression du bruit et normalisation :

— Suppression des éléments non informatifs (messages bot, templates vides, duplicatas)

— Normalisation des encodages et formats

— Extraction du texte utile depuis des structures complexes (HTML, Markdown)

— Remplacement des éléments non textuels (images → markers)

Critère de qualité : ratio signal/bruit élevé.

4.3.3 3. Enrichissement (Feature Engineering)

Ajout de métadonnées structurées par analyse automatique :

— **Classification** : catégoriser chaque document (type, sévérité, couche architecturale)

— **Extraction d'entités** : identifier les boards, composants, versions mentionnées

— **Scoring** : évaluer la force probante (*evidence strength*) de chaque document

— **Filtrage qualité** : marquer les documents invalides (trop courts, hors-scope, spam)

Critère de qualité : précision et rappel de la classification.

4.3.4 4. Mise en Relation (Linking & Similarity)

Création de liens entre documents :

— Similarité textuelle (TF-IDF, embeddings)

— Liens structurels (issue → PR → commit)

— Regroupement thématique (clustering)

Critère de qualité : pertinence des liens (pas de faux positifs bruyants).

4.3.5 5. Découpage (Chunking)

Division des documents longs en segments optimaux pour le retrieval :

— Respect des frontières sémantiques (paragraphes, sections)

— Taille bornée (typiquement 256–512 tokens)

— Overlap pour maintenir la cohérence contextuelle

Critère de qualité : chaque chunk est auto-suffisant et pertinent.

4.3.6 6. Validation et Export

Dernière étape avant l'indexation :

— Validation structurelle (JSON Schema)

— Vérification des champs obligatoires

— Export dans le format attendu par le système cible (KB)

— Écriture de fichiers de traçabilité (summary, manifests)

Critère de qualité : conformité 100% au schéma cible.

4.4 Notre Pipeline : Mapping Théorie → Implémentation

Le tableau suivant fait le lien entre les étapes canoniques et notre implémentation concrète :

Étape Canonique	**Stage Pipeline**	**Implémentation**
Collecte	Stage 1 (Ingestion)	`fetch_issues.py`, `fetch_files.py`, `repo_sync_service.py`

Nettoyage	Stage 2 (Cleaning)	`clean_issues.py`, `clean_files.py` (V1), `clean_files_v3.py` (V3 par type)
Enrichissement	Stage 3 (Enrichment)	`issues_to_docs_v2.py` — heuristiques NLP (layer, severity, board, component)
Mise en Relation	Stage 4 (Similarity)	`compute_issue_similarity_v2.py` — TF-IDF cosine, top-k voisins
Découpage	Stage 5 (Chunking)	`docs_to_chunks_*_v2.py` — paragraphe/section, max 512 tokens
Validation	Stage 6 (Evaluation)	`validate_schemas.py`, `test_stats_*_v2.py`
Export	Stage 7 (Delivery)	`export_st_ready_*.py` — 4 types d'artefacts JSON

4.5 Métriques de Qualité d'un Pipeline

Pour évaluer la qualité d'un pipeline de preprocessing, on utilise des métriques à chaque niveau :

$$Q_{\text{pipeline}} = f(Q_{\text{complétude}},\ Q_{\text{classification}},\ Q_{\text{enrichissement}},\ Q_{\text{conformité}}) \tag{4.1}$$

— $Q_{\text{complétude}}$: pourcentage de données source effectivement récupérées

— $Q_{\text{classification}}$: taux de couverture des champs de classification (non-null)

— $Q_{\text{enrichissement}}$: taux de succès des transformations IA (descriptions d'images)

— $Q_{\text{conformité}}$: pourcentage de documents passant la validation schéma

L'objectif est de maximiser ces métriques tout en maintenant la **traçabilité** (pouvoir remonter de tout artefact final vers sa source brute).

4.6 Patterns d'Architecture Pipeline

4.6.1 Pattern Batch vs Streaming

Notre pipeline utilise le pattern **batch** : toutes les données d'un repository sont traitées en une seule exécution. C'est adapté car :

— Les données GitHub changent lentement (quelques issues/semaine)

— Le calcul de similarité nécessite le corpus complet

— La validation globale (schéma) ne peut se faire que sur l'ensemble

4.6.2 Pattern "Rejouable par Stage"

Chaque stage lit un fichier JSON d'entrée et écrit un fichier JSON de sortie. Cela permet de :

— Relancer une étape sans tout recalculer (`-skip-ingestion`)

— Débugger une étape en isolation

— Tester de nouvelles heuristiques sur les données intermédiaires existantes

— Comparer les résultats entre deux versions d'un même stage

4.6.3 Pattern "Gate de Qualité"

Avant l'upload final vers la KB, un gate de validation bloque la progression si les données sont non conformes :

```
Pipeline OK → validate_schemas.py → PASS → Upload autorisé
                                  → FAIL → Upload bloqué, alerte
```

Ce pattern empêche la propagation de données corrompues vers la production.

Chapitre 5

Méthodologie et Architecture

5.1 Approche Méthodologique

Le projet suit une approche itérative :

1. **Phase 1** : Pipeline H7 (série pilote, 12 repos)
2. **Phase 2** : Extension multi-séries (F4, H5, U5, WL)
3. **Phase 3** : Enrichissement IA (Alfred Vision) et évaluation

5.2 Architecture Globale

Le système s'articule autour de 3 couches :

1. **Couche Data** : ingestion et stockage structuré (JSON)
2. **Couche Processing** : pipeline Python en 7 étapes
3. **Couche Delivery** : export et upload vers la KB cloud

5.3 Choix Techniques

Choix	**Justification**
Python 3.14	Écosystème riche (requests, scikit-learn), déploiement simple
JSON comme format pivot	Natif pour APIs REST, lisible, versionnable
TF-IDF (scikit-learn)	Léger, rapide, suffisant pour la similarité textuelle

Heuristiques vs ML	Déterministe, explicable, pas de données d'entraînement requises
Architecture modulaire	Chaque étape est rejouable indépendamment

5.4 Diagramme de Flux de Données

La Figure 5.1 illustre la vue globale du pipeline et le flux de données entre les 7 stages.

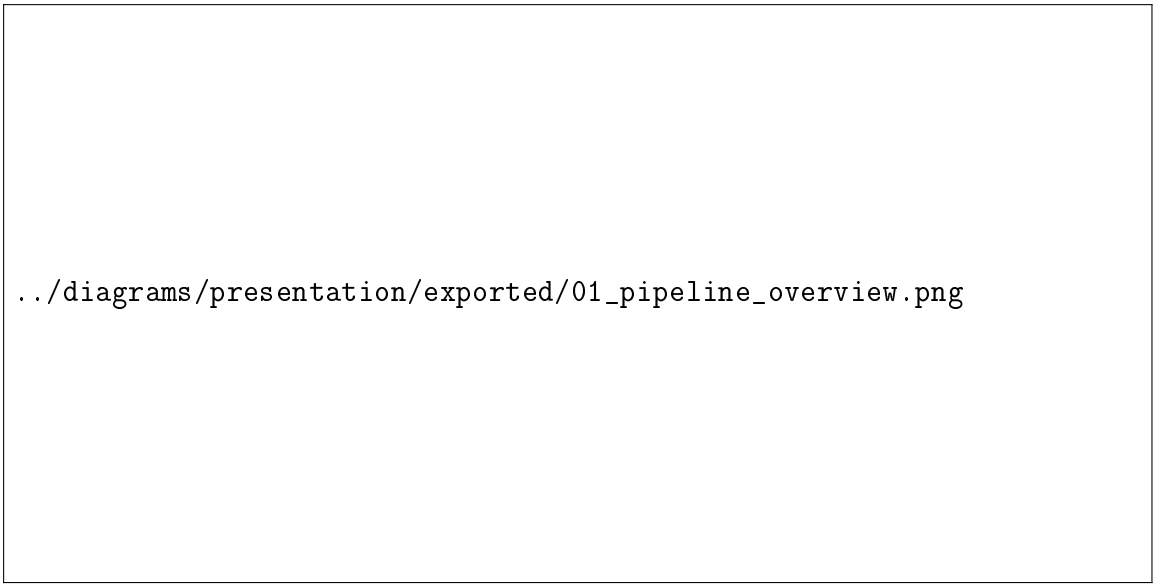


FIGURE 5.1 – Vue d'ensemble du pipeline de preprocessing (7 stages)

La Figure 5.2 détaille les connexions entre stages, les fichiers intermédiaires produits à chaque étape, et le branchement de l'enrichissement Alfred Vision AI.

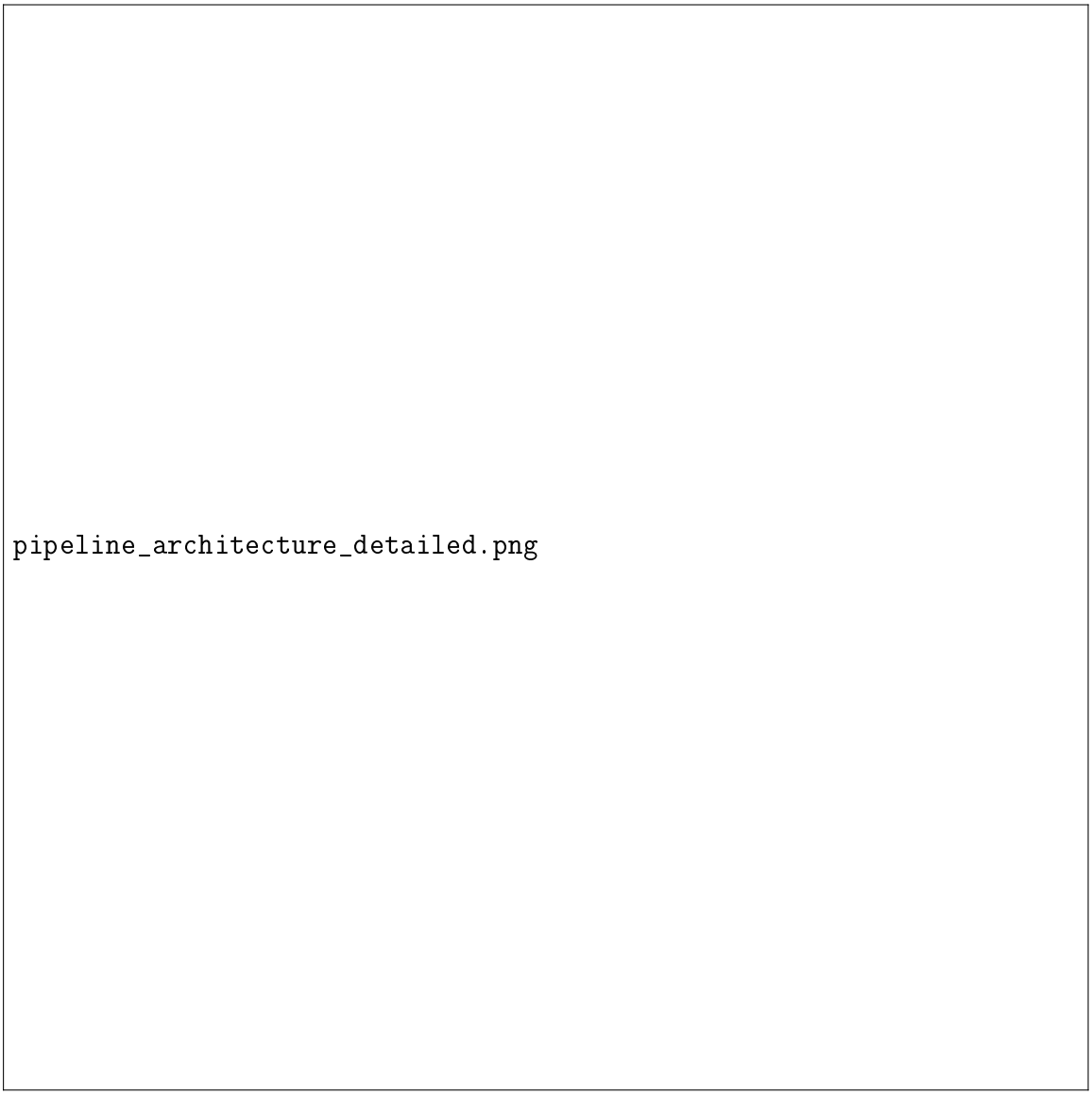


FIGURE 5.2 – Architecture détaillée du pipeline : stages, fichiers intermédiaires et branchement Alfred (draw.io)

Le diagramme montre le flux : GitHub (25 repos) → Ingestion (raw_*) → Cleaning V1/V3 (clean_*) → Enrichment (docs_*) → Similarity (_sim) → Validation → Delivery (st_ready_*) → Upload KB #793 → ST ChatGPT. Alfred Vision intervient en parallèle pour décrire les images des issues et les figures PDF.

5.5 Flux Source-to-Answer

La Figure 5.3 montre le parcours complet d'une donnée depuis GitHub jusqu'à la réponse au client final.



FIGURE 5.3 – Flux de données end-to-end : de la source GitHub à la réponse utilisateur

5.6 Architecture de Delivery et Upload KB



FIGURE 5.4 – Architecture de livraison et upload vers la Knowledge Base #793

Chapitre 6

Implémentation

6.1 Ingestion (Stage 1)

6.1.1 Objectif

Récupérer exhaustivement toutes les données des repositories GitHub configurés pour une série MCU donnée. La configuration est définie dans un fichier `config_XX.json` qui liste les repos à ingérer.

6.1.2 Mécanisme

L'ingestion s'appuie sur 5 scripts spécialisés :

— `fetch_issues.py` : récupère toutes les issues (ouvertes + fermées) avec leurs commentaires, via l'API GitHub REST avec pagination automatique (100 items/page)

— `fetch_prs.py` : récupère les Pull Requests et leurs reviews

— `fetch_commits.py` : récupère l'historique des commits pour le linking Issue→PR→Commit

— `fetch_issue_pr_commit_links.py` : construit les chaînes de traçabilité croisées

— `repo_sync_service.py` : clone/met à jour les repos localement pour l'extraction de fichiers

— `fetch_files.py` : parcourt l'arborescence des repos clonés et extrait le contenu des fichiers pertinents (README, Release Notes, sources, PDFs)

Gestion du rate-limiting : l'API GitHub autorise 5 000 requêtes/heure avec un token d'authentification. Le pipeline implémente un sleep adaptatif (0,2s entre les issues, 0,1s entre les commentaires) et un retry exponentiel en cas d'erreur 403/429.

6.1.3 Exemple de Sortie — Issue Brute

Listing 6.1 – Extrait de raw_issues_stm32cubeh7.json (issue brute)

```
{
  "number": 42,
  "title": "HAL_SPI_TransmitReceive_DMA fails on STM32H743",
  "state": "closed",
  "labels": ["bug"],
  "body": "When using HAL_SPI_TransmitReceive_DMA with...",
  "comments": [
    {"user": "dev123", "body": "Same issue on NUCLEO-H743ZI..."},
    {"user": "st-support", "body": "Fixed in v1.11.0..."}
  ],
  "created_at": "2023-06-15T10:30:00Z",
  "closed_at": "2023-08-20T14:00:00Z"
}
```



FIGURE 6.1 – Détail de l'étape d'ingestion : scripts et sorties (draw.io)

6.2 Nettoyage (Stage 2)

Le nettoyage prépare les données brutes pour l'enrichissement en éliminant le bruit textuel :

6.2.1 Nettoyage des Issues

— Normalisation du texte (retours chariot, backticks, espaces)

— Extraction des images en markers `[IMAGE ATTACHED]` avec collecte des URLs

— Structuration : assemblage Title + Body + Comments en un seul champ `clean_text`

— Conservation des métadonnées brutes (labels, dates, état)

6.2.2 Exemple JSON — Issue Après Nettoyage

Listing 6.2 – Extrait de clean_issues_stm32cubeh7.json (issue nettoyée)

```
{
  "number": 42,
  "title": "HAL_SPI_TransmitReceive_DMA fails on STM32H743",
  "state": "closed",
  "labels": ["bug"],
  "clean_text": "HAL_SPI_TransmitReceive_DMA fails on STM32H743.
     When using HAL_SPI_TransmitReceive_DMA with circular mode and
     cache enabled, a hard fault occurs after the second transfer.
     [IMAGE ATTACHED: screenshot_debugger.png] Workaround: disable
     D-cache before SPI init. Same issue on NUCLEO-H743ZI with
     CubeH7 v1.10.0. Fixed in v1.11.0 by adding cache maintenance
     in HAL_SPI_TxRxCpltCallback.",
  "images_count": 1,
  "image_urls": ["https://user-images.githubusercontent.com/xxx/
     screenshot_debugger.png"],
  "created_at": "2023-06-15T10:30:00Z",
  "closed_at": "2023-08-20T14:00:00Z"
}
```

Transformation clé : le champ `clean_text` fusionne le titre, le body et tous les commentaires de l'issue en un seul texte contigu, prêt pour l'enrichissement NLP. Les images sont remplacées par des markers textuels avec leurs URLs collectées séparément pour l'enrichissement Alfred Vision.

6.2.3 Nettoyage des Fichiers

— Lecture du contenu (UTF-8, fallback Latin-1)

— Extraction du texte des PDFs (pypdf + rendu pages + détection figures OpenCV)

— Construction de l'URL GitHub pour chaque fichier

— Filtrage : seuls les fichiers pertinents sont retenus (README, Release Notes, source technique, documentation PDF)

6.2.4 Exemple JSON — Fichier Après Nettoyage

Listing 6.3 – Extrait de clean_files_stm32h7xx-hal-driver.json


```
{
  "path": "Src/stm32h7xx_hal_spi.c",
  "repo": "stm32h7xx-hal-driver",
  "file_type": "source_code",
  "url": "https://github.com/STMicroelectronics/stm32h7xx-hal-
    driver/blob/master/Src/stm32h7xx_hal_spi.c",
  "original_size": 89247,
  "clean_size": 80322,
  "is_valid": true,
  "clean_text": "HAL_StatusTypeDef HAL_SPI_Init(SPI_HandleTypeDef *
    hspi) { ... } HAL_StatusTypeDef HAL_SPI_TransmitReceive_DMA
    (...) { ... }"
}
```


Transformation clé : le bloc de licence en-tête (copyright STMicroelectronics, 20–40 lignes) est supprimé par `strip_license()`, réduisant le bruit de ∼10%. Le champ `is_valid` passe à `false` si le texte nettoyé fait moins de 100 caractères.

Note importante : Le nettoyage ne fait **aucune classification** (pas de détection de layer, board, component). Cette tâche est réalisée à l'étape suivante (Enrichissement).

6.3 Nettoyage V3 — Nettoyage Intelligent par Type (Stage 2b)

Le nettoyage V1/V2 appliquait une stratégie uniforme à tous les fichiers. Or les sources ingérées sont hétérogènes : Release Notes en HTML, README en Markdown avec badges, fichiers source C avec licences volumineuses, et headers CMSIS de plus d'1 Mo. La version 3 introduit un **nettoyage par type** qui adapte la stratégie au contenu réel de chaque fichier.

6.3.1 Stratégies de Nettoyage par Type

Le script `clean_files_v3.py` route chaque fichier vers une stratégie spécialisée :

Type détecté	Stratégie	Transformations appliquées
Release Notes	`strip_html()`	Suppression balises HTML, conservation texte pur, normalisation espaces

README / .md	`clean_markdown()`	Suppression badges CI, liens images, normalisation titres
Source code (.c/.h)	`clean_source_code()`	Suppression bloc licence (copyright), conservation code technique
CMSIS device header	`filter_cmsis_device_header()`	Filtrage intelligent (voir ci-dessous)
Autres	Inchangé	Aucune transformation

La Figure 6.2 représente l'arbre de décision du routage V3, avec les taux de réduction mesurés pour chaque stratégie.



FIGURE 6.2 – Arbre de décision du nettoyage V3 : routage par type de fichier avec taux de réduction (draw.io)

L'arbre de décision route chaque fichier via `detect_file_type()` *vers la stratégie*

adaptée : release_notes → strip_html() (−33%), readme → clean_md() (−15%), source → strip_license() (−10%), cmsis >100 Ko → filter_cmsis() (−96%), other → pass-through.

6.3.2 Filtrage CMSIS Device Headers

Les fichiers de type `stm32*xx.h` (CMSIS device) sont les plus volumineux du corpus (1–1.7 Mo). Ils contiennent principalement des définitions de registres bit-à-bit inutiles pour le RAG. Le filtre V3 ne conserve que les sections à valeur diagnostique :

1. **Énumération IRQ** : liste des vecteurs d'interruption (`typedef enum IRQn`)
2. **Adresses de base** : adresses mémoire des périphériques (`#define PERIPH_BASE`)
3. **Structures TypeDef** : définitions de registres par périphérique (`typedef struct {...}`)

Seuil d'activation : le filtre ne s'applique qu'aux fichiers dépassant 100 Ko (les petits headers sont conservés intégralement).

6.3.3 Résultat du Filtrage — Exemple JSON

Voici un extrait du fichier `clean_files_cmsis_device_h7_v3.json` montrant un header CMSIS après filtrage V3 :

Listing 6.4 – Header CMSIS après filtrage V3 (extrait)


```
{
  "path": "Include/stm32h743xx.h",
  "repo": "cmsis_device_h7",
  "file_type": "cmsis_device_header",
  "original_size": 1458301,
  "clean_size": 52140,
  "reduction_pct": 96.4,
  "is_valid": true,
  "clean_text": "typedef enum { NonMaskableInt_IRQn = -14, ... }
     IRQn_Type; #define PERIPH_BASE 0x40000000UL ... typedef struct
      { __IO uint32_t CR1; __IO uint32_t CR2; ... } SPI_TypeDef;"
}
```


Le fichier passe de 1 458 301 caractères à 52 140 (−96, 4%), ne conservant que les IRQ, adresses de base et TypeDef.

6.3.4 Résultats du Nettoyage V3

Fichier	Taille V2	Taille V3	Réduction
`stm32h503xx.h`	1 074 499	36 499	−96.6%
`stm32h562xx.h`	1 722 072	65 470	−96.2%
`stm32h743xx.h`	1 458 301	52 140	−96.4%
Release Notes (moy.)	45 200	30 300	−33.0%
Sources avec licence	12 800	11 500	−10.2%

Impact global : Sur les 47 repos (5 séries MCU), le volume total des fichiers nettoyés est réduit de **38%** en moyenne, ce qui améliore directement la densité d'information des chunks et la pertinence du retrieval.

6.3.5 Compatibilité et Non-Régression

Le V3 n'écrase jamais les sorties V1/V2 :

— Fichiers d'entrée : `clean_files_<repo>.json` (V1) ou `clean_files_<repo>_v3.json` (V3)

— Les étapes aval (enrichissement, delivery) acceptent un flag `-v3` pour router automatiquement vers les fichiers V3

— Les datasources V3 sont uploadées vers des IDs séparés (29041–29045)

6.4 Enrichissement V2 (Stage 3)

6.4.1 Heuristiques de Classification

Le module `stm32cube_preprocessing_heuristics.py` applique des règles basées sur :

— **Regex patterns** : détection de séries MCU, composants, boards

— **Analyse des labels** : mapping label→layer (bug→bug_report, etc.)

— **Scoring pondéré** : longueur du body, présence de code, de logs d'erreur

6.4.2 Exemple de Classification

```
{
  "issue_number": 42,
  "title": "HAL_SPI_TransmitReceive DMA fails on STM32F446",
  "layer": "HAL",
```

```
  "component": "SPI",
  "board": "NUCLEO-F446RE",
  "issue_kind": "bug_report",
  "severity": "high",
  "evidence_strength": "high"
}
```

6.5 Similarité (Stage 4)

6.5.1 Algorithme

1. Construction de la matrice TF-IDF sur le corpus (titre + body + commentaires) :

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{|\{d' \mid t \in d'\}|} \tag{6.1}$$

2. Calcul de la similarité cosinus paire-à-paire :

$$\text{sim}(d_i, d_j) = \frac{\vec{v}_i \cdot \vec{v}_j}{\|\vec{v}_i\| \times \|\vec{v}_j\|} \tag{6.2}$$

3. Sélection des k plus proches voisins avec seuil $\theta = 0.3$:

$$\text{related}(d_i) = \{d_j \mid \text{sim}(d_i, d_j) \geq \theta,\ j \neq i\} \tag{6.3}$$

6.5.2 Exemple JSON — Issue avec Liens de Similarité

Listing 6.5 – Extrait de docs_issues_stm32cubeh7_v2_sim.json (avec voisins)

```
{
  "issue_number": 42,
  "title": "HAL_SPI_TransmitReceive_DMA fails on STM32H743",
  "similar_issues": [
    {"issue_number": 78, "similarity": 0.72, "title": "SPI DMA
       circular mode crash H750"},
    {"issue_number": 156, "similarity": 0.45, "title": "DMA cache
       coherency issue H743"},
    {"issue_number": 203, "similarity": 0.38, "title": "Hard fault
       after second SPI transfer"}
  ],
  "related_count": 3
}
```

Paramètres : le seuil $\theta = 0.3$ élimine les liens trop faibles (bruit). Sur H7, seulement 22% des issues ont au moins un voisin au-dessus de ce seuil, ce qui reflète la diversité des problèmes (chaque issue traite un bug unique). Le nombre k de voisins n'est pas borné — toutes les paires dépassant θ sont conservées.



FIGURE 6.3 – Détail du processus d'enrichissement et de calcul de similarité

6.6 Chunking V2 (Stage 5 — Optionnel)

Note importante : le chunking est réalisé localement à des fins de **test et validation** uniquement. En production, la plateforme ST ChatGPT gère elle-même le découpage des documents lors de l'indexation (paramètre `json-split-size: 1000`). Le chunking local nous permet de :

— Vérifier la taille et la qualité des segments avant upload

— Tester la recherche TF-IDF en local (évaluation préliminaire)

— Comparer les stratégies de découpage sans impacter la KB en production

Stratégie de découpage local :

— Issues : chunks par paragraphe, max 512 tokens, overlap 50 tokens

— Files : chunks par section (README headers, fonctions C)

Le flag `-skip-chunking` dans le workflow permet de sauter cette étape pour les runs de production, car la plateforme gère le chunking automatiquement à l'upload.

6.7 Delivery (Stage 7)

6.7.1 Resolver Cases

Générés à partir d'issues fermées avec PRs liées :

1. Identification de la chaîne Issue → PR → Commits via `fetch_issue_pr_commit_links.py`
2. Évaluation du `resolution_status` :
   — `merged_fix` : PR mergée avec fichiers modifiés identifiés (confiance haute)
   — `candidate_fix` : PR liée mais non mergée ou lien faible (confiance moyenne)
   — `reference_only` : issue résolue sans PR formelle (information contextuelle)
3. Construction du texte structuré : Problème → Signal → Évaluation → Preuves

6.7.2 Exemple JSON — Resolver Case

Listing 6.6 – Extrait de st_ready_resolver_stm32cubeh7.json

```
{
  "issue_number": 42,
  "repo": "stm32cubeh7",
  "label": "SPI DMA TransmitReceive fails on STM32H743 in circular
     mode",
  "resolution_status": "merged_fix",
  "link_confidence": "high",
  "primary_pr_number": 87,
  "primary_changed_file": "Src/stm32h7xx_hal_spi.c",
  "resolver_card_text": "PROBLEM: HAL_SPI_TransmitReceive_DMA
     causes hard fault after second transfer in circular mode.
     SIGNAL: D-cache stale read on DMA buffer. EVALUATION: Cache
     maintenance missing in TxRxCpltCallback. FIX: Added
     SCB_InvalidateDCache_by_Addr() call in
     HAL_SPI_TxRxCpltCallback. PROOF: PR #87 merged 2023-08-15,
     verified on NUCLEO-H743ZI.",
  "impacted_modules": ["SPI", "DMA"],
  "impacted_boards": ["NUCLEO-H743ZI"]
}
```

Valeur ajoutée : contrairement aux issues brutes, le Resolver Case fournit la **chaîne de preuves complète** (issue → PR #87 → fichier modifié `stm32h7xx_hal_spi.c`) en un seul document. L'ingénieur peut vérifier le fix exact en suivant les liens.

6.7.3 Diagnostic Cards

Cartes de diagnostic structurées pour les problèmes récurrents :

1. Sélection des issues à haute valeur diagnostique (issues fermées avec root cause identifié, `evidence_strength` ≥ `medium`)
2. Génération : Symptôme → Cause Racine → Résolution → Validation
3. Ajout de **query aliases** (reformulations multiples du problème) pour améliorer le recall de la recherche RAG

6.7.4 Exemple JSON — Diagnostic Card

Listing 6.7 – Extrait de st_ready_diagnostic_stm32cubeh7.json

```
{
  "diagnostic_id": "DIAG-H7-042",
  "repo": "stm32cubeh7",
  "issue_number": 42,
  "symptom": "Hard fault occurs after second SPI DMA transfer in
     circular mode on STM32H743",
  "root_cause": "D-cache not invalidated before DMA reception. CPU
     reads stale data from cache instead of DMA-updated SRAM.",
  "resolution": "Call SCB_InvalidateDCache_by_Addr() before reading
      SPI RX buffer, or place DMA buffers in non-cacheable SRAM
     region (MPU configuration).",
  "affected_versions": ["CubeH7 v1.9.0", "CubeH7 v1.10.0"],
  "fixed_version": "CubeH7 v1.11.0",
  "query_aliases": [
    "SPI DMA hard fault STM32H7",
    "cache coherency SPI circular mode",
    "HAL_SPI_TransmitReceive_DMA crash",
    "D-cache DMA corruption H743"
  ],
  "evidence_strength": "high",
  "impacted_boards": ["NUCLEO-H743ZI", "STM32H743I-EVAL"]
}
```

Structure clé : les `query_aliases` sont le mécanisme principal d'amélioration du recall. Lorsqu'un utilisateur pose la question "mon SPI crash après le 2e transfert DMA", la recherche hybride retrouve cette Diagnostic Card grâce aux alias qui couvrent différentes formulations du même problème. C'est cette structure qui explique le saut de QI de 61 à 94.



FIGURE 6.4 – Génération des Resolver Cases et Diagnostic Cards

La Figure 6.5 synthétise les relations entre les 4 types d'artefacts, leurs sources dans le pipeline, et leur destination dans la KB.

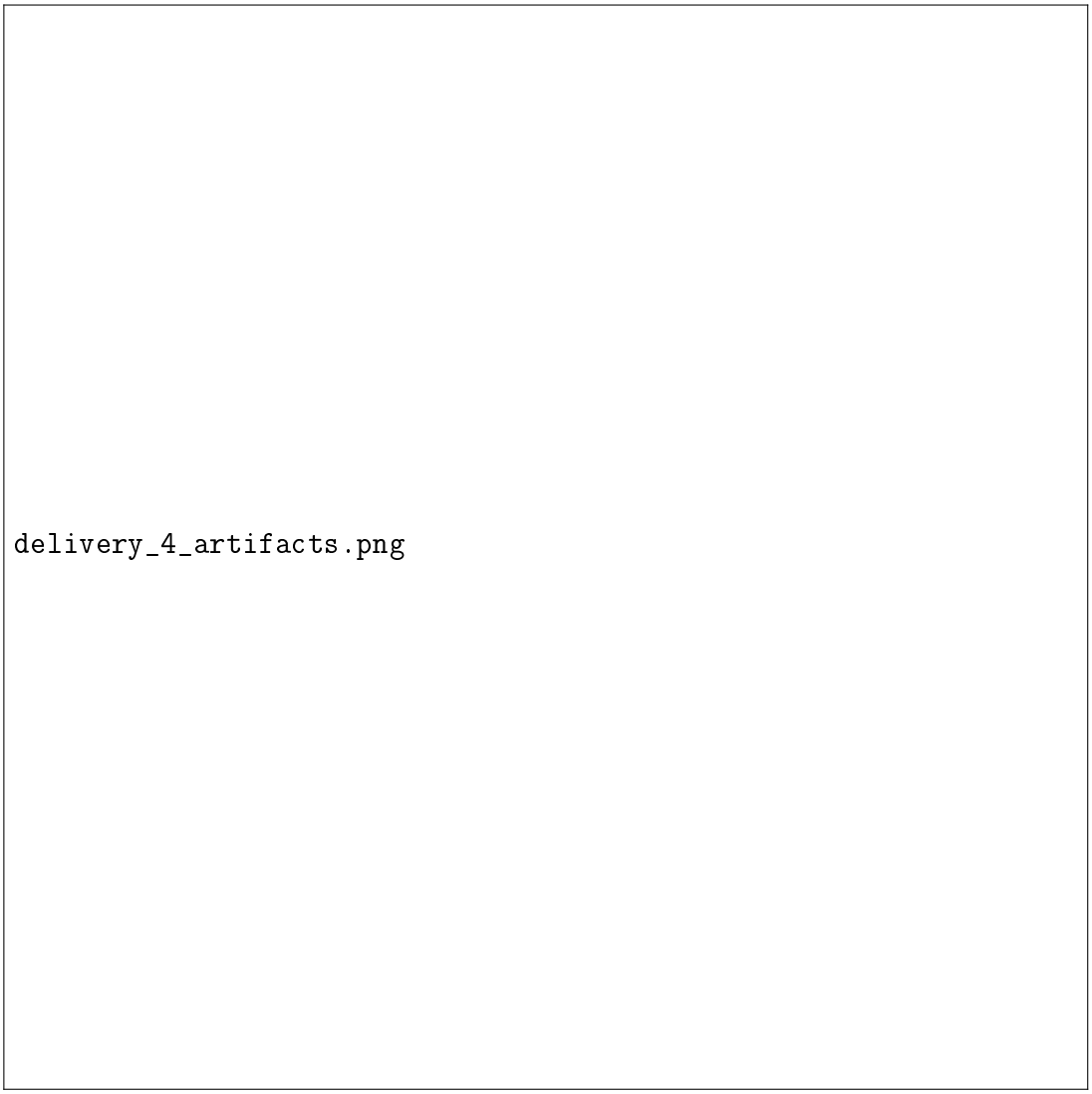


FIGURE 6.5 – Les 4 types d'artefacts delivery et leurs relations vers la KB #793 (draw.io)

Le diagramme montre les 4 types d'artefacts : Issues Structurées (∼2 000 docs), Files Documentés (∼14 000 docs), Resolver Cases (∼390 docs) et Diagnostic Cards (∼157 docs), tous alimentés vers la KB #793 (38 datasources). Les Resolver Cases et Diagnostic Cards sont dérivés des Issues via les champs issue_ number et metadata.

6.7.5 Alfred Vision — Enrichissement d'Images

De nombreuses issues GitHub contiennent des captures d'écran (erreurs IDE, oscillogrammes, configurations CubeMX). Ces images sont opaques pour la recherche textuelle RAG.

Le pipeline intègre **Alfred Vision AI**, un service LLM interne ST, pour convertir automatiquement les images en descriptions textuelles :

1. Le script d'export génère des markers `[IMAGE ATTACHED]` dans le texte

2. Les URLs des images sont extraites dans le champ `image_urls`
3. Alfred Vision télécharge chaque image et produit une description sémantique
4. Le marker est remplacé par `[IMAGE DESCRIPTION] ...`

Résultats H7 : 97/98 images décrites avec succès (taux de succès 98.9%).

Fix appliqué : La fonction `first_sentence()` supprime les markers `[IMAGE ATTACHED]` du résumé `root_cause_hint` pour éviter la duplication des markers entre les sections "Root cause" et "Key comments", ce qui causait des descriptions `UNAVAILABLE` par épuisement des URLs.



FIGURE 6.6 – Intégration Alfred Vision AI pour la description automatique d'images

6.8 Architecture Agents — Persona ST ChatGPT

Le système final s'articule autour d'une architecture **multi-agents** où chaque composant a un rôle bien défini.

6.8.1 Vue d'ensemble



FIGURE 6.7 – Architecture multi-agents du système ST GitHub Analyzer (draw.io)

Le diagramme montre le flux : Utilisateur (Ingénieur STM32) → Persona ST GitHub Analyzer → KB Tool (Unstructured, recherche hybride) avec fallback vers NovaX Tool (si la KB ne retourne aucun résultat pertinent). Le Pipeline Preprocessing alimente la KB depuis GitHub STMicroelectronics via des uploads JSON.

6.8.2 Persona : ST GitHub Analyzer

La persona est un agent conversationnel déployé sur la plateforme **ST ChatGPT** (Azure OpenAI interne). Elle est configurée avec :

Paramètre	**Valeur / Description**
Nom	ST GitHub Analyzer

Modèle	GPT-4o (Azure OpenAI)
Température	0.2–0.3 (réponses factuelles)
KB associée	KB #793 (ST GitHub Analyzer)
Tool principal	ST-GitHub-Analyzer-Unstructured (Hybrid Search)
Tool fallback	NovaX (STM32 Bugzilla + STCommunity Knowledge Articles)
Search type	Hybrid (Semantic + Full-text)
Semantic threshold	0.55
Max documents	20 (semantic) + 12 (full-text)

6.8.3 Tool 1 : KB Unstructured (Recherche Hybride)

C'est le tool **principal**. Il interroge la KB #793 qui contient les artefacts générés par le pipeline :

- **Issues** : cartes structurées avec métadonnées, symptôme, root cause, fix
- **Files** : documentation HAL/BSP/CMSIS extraite des repositories
- **Resolver Cases** : chaînes Issue → PR → Commit avec preuves
- **Diagnostic Cards** : analyses de pannes structurées (symptôme → hypothèses → validation)

La recherche hybride combine :

1. **Semantic** : compréhension du sens ("mon SPI ne marche plus" → retrouve les issues SPI DMA)
2. **Full-text** : correspondance exacte ("HAL_SPI_Transmit_DMA" → match direct)

6.8.4 Tool 2 : NovaX Fallback

Quand la KB ne contient pas de réponse pertinente, la persona peut appeler **NovaX**, un agent STM32 basé sur STM32 Bugzilla et STCommunity Knowledge Articles.

Configuration critique — La description du tool doit **forcer la priorité KB** :

> *"I am a fallback assistant for STM32 firmware questions that are NOT covered by the Knowledge Base. The Knowledge Base is ALWAYS the primary source, search it FIRST for every question. Use this tool ONLY when the Knowledge Base returned no relevant result."*

Risque identifié : Si la description est trop permissive, la persona appelle NovaX au lieu de chercher dans la KB → réponses générales non ancrées dans les artefacts GitHub

→ dégradation du Quality Index. NovaX doit rester un secours pour les questions hors couverture KB.

6.8.5 Stratégie de Réponse (System Prompt)

Le system prompt de la persona impose une stratégie en 4 cas :

1. **Match direct KB** → citer la source avec URL GitHub
2. **Match partiel** → diagnostic + workaround candidat + étapes de validation
3. **Question configuration générale** → explication technique directe
4. **Aucun match** → appeler Alfred OU proposer une action concrète (jamais "je ne sais pas")

6.8.6 Pipeline comme Agent de Maintenance

Le pipeline de preprocessing joue lui-même le rôle d'un **agent de maintenance** de la KB :

Rôle	**Description**
Ingestion périodique	Détecte les nouvelles issues et mises à jour GitHub (bi-mensuel)
Enrichissement	Classifie automatiquement par heuristiques NLP (layer, component, severity)
Validation qualité	Gate de schéma bloquant les uploads de données corrompues
Mise à jour KB	Delete + Re-add automatique via GitHub Actions
Évaluation continue	Quality Index 52 questions pour détecter les régressions

Chapitre 7

Configuration de la Plateforme ST ChatGPT

Ce chapitre détaille la configuration de la persona **ST GitHub Analyzer** sur la plateforme ST ChatGPT, les justifications de chaque paramètre, et l'évolution incrémentale du Quality Index qui a guidé nos choix.

7.1 Architecture de la Plateforme

La plateforme ST ChatGPT est un service Azure OpenAI interne déployé par STMicroelectronics. Elle permet de créer des **personas** — des agents conversationnels spécialisés — équipés de deux types d'outils :

— **KB (Knowledge Base)** : recherche dans une base de connaissances structurée

— **Persona-as-a-Tool** : appel à un autre agent comme fallback

7.2 Configuration du Tool KB

7.2.1 Paramètres Retenus

Paramètre	**Valeur choisie**	**Alternatives testées**
KB Type	Unstructured	Structured (rejeté)
Search Type	**Hybrid**	Semantic seul, Full-text seul
Semantic Threshold	**0.55** (S0 courant) / **0.45** (S1 candidat)	0.55, 0.45, 0.65

Semantic Max Doc Count	**25** (S0) / **35** (S1)	15, 18, 25, 35
Full-text Max Doc Count	**12**	5, 8, 20
Tool fallback	**NovaX** restrictif	Fallback permissif (rejeté)
Modèle LLM	GPT-5.1	GPT-4o (précédent)

Lecture opérationnelle : la configuration S0 reste le point d'équilibre courant, car elle combine une bonne stabilité (QI=65) et un volume documentaire maîtrisé. La configuration S1 obtient le meilleur score mesuré (QI=66) en augmentant le rappel ; elle est donc le candidat prioritaire pour la configuration finale, sous réserve de validation du test complet en cours.

7.2.2 Justification : Recherche Hybride

La recherche **hybride** (Semantic + Full-text) est supérieure aux deux modes isolés dans notre contexte :

Mode	**Force**	**Faiblesse pour STM32**
Semantic seul	Comprend le sens ("mon SPI marche pas" → issues SPI)	Rate les noms exacts d'API (`HAL_SPI_Transmit_DMA`)
Full-text seul	Match exact sur les identifiants techniques	Rate les reformulations ("erreur Ethernet" vs "LwIP crash")
Hybrid	Combine les deux	Complexité légèrement supérieure

Résultat mesuré : sur le benchmark complet de 96 questions, la recherche hybride surpasse les configurations mono-mode. Le semantic-only (S5) atteint 41/100 et le fulltext-heavy (S4) 47/100, alors que les stratégies hybrides équilibrées atteignent 63–66/100. Le résultat confirme que les questions STM32 combinent deux signaux complémentaires : des formulations naturelles ("problème LoRa join", "DMA corruption") et des identifiants exacts (`HAL_SPI_Transmit_DMA`, `NUCLEO-H743ZI`, noms de fichiers). Une stratégie purement sémantique perd les ancres exactes ; une stratégie trop full-text perd les reformulations.

7.2.3 Justification : Seuil Sémantique à 0.55

Le **seuil sémantique** (Semantic Threshold) est le score minimum de similarité qu'un document doit atteindre pour être retourné par la recherche sémantique. Un score de 1.0

signifie une correspondance parfaite, 0.0 aucune similarité. Ce seuil contrôle directement le compromis entre **rappel** (ne rien rater) et **précision** (pas de bruit).

Le seuil de 0.55 a d'abord été calibré empiriquement sur les campagnes H7. La campagne multi-séries de juin 2026 a ensuite comparé six stratégies sur 96 questions :

Stratégie	**Search**	**Thr.**	**Sem. docs**	**FT docs**	**QI**	**Interprétation**
S0 Current	Hybrid	0.55	25	12	65	Équilibre précision/rappel courant
S1 Recall+	Hybrid	0.45	35	15	**66**	Meilleur score ; capte plus de cas marginaux
S2 Precision+	Hybrid	0.65	18	8	63	Moins de bruit mais plus de faux négatifs
S3 Semantic-heavy	Hybrid	0.55	35	6	51	Trop peu d'ancrage exact API/fichiers
S4 Fulltext-heavy	Hybrid	0.55	15	20	47	Trop dépendant des mots exacts
S5 Semantic-only	Semantic	0.55	25	0	41	Ablation : perte forte du full-text
S6 Fulltext-only	Full-text	–	0	20	N/A	Test final en cours au 01/07/2026

Conclusion : le meilleur score obtenu est S1 (66/100), mais l'écart avec S0 est faible (+1 point). S1 augmente le rappel en abaissant le seuil à 0.45 et en retournant davantage de documents (35 sémantiques + 15 full-text). Cette stratégie est utile lorsque les questions sont implicites ou utilisent peu de noms d'API exacts. S0 reste plus prudent pour la production car il limite mieux le bruit contextuel et conserve une latence plus faible.

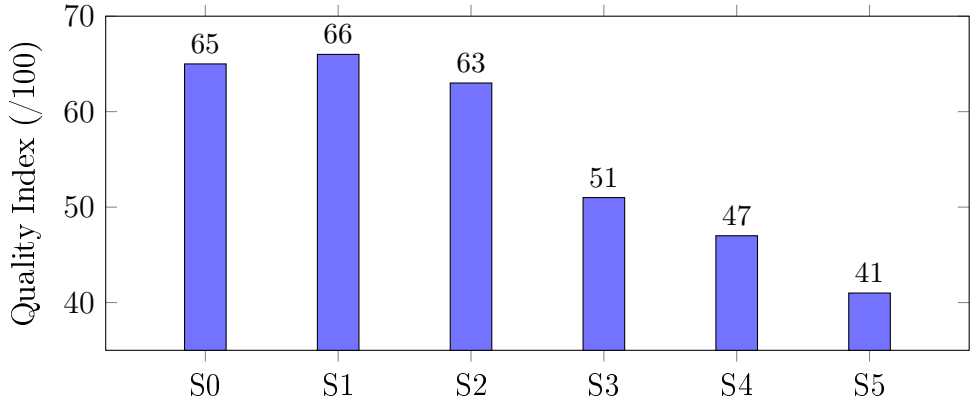


FIGURE 7.1 – Quality Index des stratégies Unstructured KB sur 96 questions — S1 maximise le rappel, S0 reste le point d'équilibre courant

7.2.4 Justification : 20 Documents Sémantiques + 12 Full-text

Le paramètre **Max Doc Count** limite le nombre de documents retournés par chaque mode de recherche. Ce nombre est injecté dans le contexte du LLM, qui doit ensuite synthétiser une réponse à partir de ces documents. Un nombre trop élevé sature la fenêtre de contexte et ralentit la réponse ; un nombre trop bas risque de manquer le document pertinent.

— **25 documents sémantiques (S0)** : couvre les reformulations et les questions indirectes tout en gardant le contexte compact. C'est le réglage courant stable.

— **35 documents sémantiques (S1)** : augmente le rappel pour les questions implicites ou mal formulées. Le gain observé est réel mais limité (+1 point), ce qui montre que le rappel supplémentaire apporte aussi du bruit.

— **12 à 15 documents full-text** : assure que les correspondances exactes d'API et de noms de fichiers sont toujours retrouvées. Ce canal est indispensable : le score S5 semantic-only chute à 41/100, preuve que la branche full-text porte une partie majeure de la valeur dans le support STM32.

Total de documents par requête : S0 retourne jusqu'à $25 + 12 = 37$ documents, et S1 jusqu'à $35 + 15 = 50$ documents. S1 améliore le rappel mais consomme plus de contexte ; il doit donc être accompagné d'un prompt de synthèse strict pour éviter que le LLM ne mélange plusieurs séries ou périphériques.

7.2.5 Définition : Search Type (Hybrid)

Le **Search Type** détermine l'algorithme de recherche utilisé pour interroger la Knowledge Base. La plateforme ST ChatGPT supporte trois modes :

1. **Semantic** : transforme la requête en vecteur (embedding) et recherche les documents les plus proches dans l'espace vectoriel. Comprend le *sens* de la question mais peut rater les termes exacts.
2. **Full-text** : recherche par correspondance exacte de mots (BM25/TF-IDF). Trouve les identifiants exacts (`HAL_SPI_TransmitReceive_DMA`) mais ne comprend pas les reformulations.
3. **Hybrid** : exécute les deux recherches en parallèle et fusionne les résultats (union avec déduplication). Combine la compréhension sémantique et la précision lexicale.

Le mode **Hybrid** a été retenu car les questions des ingénieurs STM32 mélangent systématiquement du langage naturel ("mon SPI crash au deuxième transfert") et des identifiants techniques exacts (`HAL_SPI_Transmit_DMA`, `NUCLEO-H743ZI`).

7.2.6 Tool Description

La description du tool KB est critique car elle guide le LLM dans sa décision d'appeler ou non le tool :

```
Retrieve technical knowledge about STM32Cube microcontroller firmware from
GitHub repositories. Contains: resolved issues with root cause and fix,
HAL/BSP source file documentation, resolver cases (step-by-step troubleshooting),
and diagnostic cards (structured fault analysis). Covers series: STM32F4,
STM32H5, STM32H7, STM32U5, STM32WL and their driver sub-repos (HAL, CMSIS,
BSP). Query this tool when the user asks about: peripheral configuration
problems, HAL API behavior, known bugs and workarounds, board-specific issues,
firmware package contents, low-power modes, communication protocols, DMA/interrupt
issues, or build errors. Use specific technical terms in queries: peripheral
names (SPI, I2C, UART, FDCAN, OSPI), HAL function names, error descriptions,
board references, and STM32 part numbers.
```

Points clés de cette description :

1. **Énumération explicite du contenu** (issues, files, resolver, diagnostic) → le LLM sait quoi chercher
2. **Liste des séries couvertes** → évite les appels pour des séries absentes (L4, G4)
3. **Instruction de reformulation** ("Use specific technical terms") → améliore la qualité des queries envoyées à la KB
4. **Exemples de cas d'usage** → le LLM sait *quand* appeler le tool vs répondre directement

7.3 Configuration du Tool NovaX (Fallback)

7.3.1 Problème Identifié

Lors de l'ajout initial d'un fallback généraliste, le Quality Index a chuté lorsque la persona appelait le fallback au lieu de chercher dans la KB. Le problème n'était pas le modèle lui-même, mais le **routage** : une description trop large encourage le LLM principal à déléguer trop tôt, ce qui remplace des réponses traçables issues de la KB par des réponses générales.

> `[Description initiale, trop permissive]` "I can help with STM32 questions. Forward the user's question."

7.3.2 Fix : NovaX comme Fallback Strictement Secondaire

Le fallback utilisé est désormais **NovaX**, un agent STM32 basé sur STM32 Bugzilla et STCommunity Knowledge Articles. Sa description force explicitement la priorité de la KB Unstructured :

> I am an STM32 MCUs smart agent based on STM32 Bugzilla & STCommunity Knowledge Articles : MPU : STM32MP1 & MP2 series ; Wireless : STM32WB, WBA, WL series ; Ultra-lowpower : STM32L0, L1, L4, L4+, L5, U0, U5 series ; Mainstream : STM32F0, G0, C0, F1, F3, G4 and C5 series ; High Performance : STM32F2, F4, H5, F7, H7 and N6 series ; Tools : STM32CubeMX, IDE, Programmer ; X-Cube : Graphics, MC, Safety, Audio, EdgeAI. I am a fallback assistant for STM32 firmware questions that are NOT covered by the Knowledge Base. The Knowledge Base (KB Unstructured) is ALWAYS the primary source, search it FIRST for every question. Use this tool ONLY when the Knowledge Base returned no relevant result or explicitly lacks coverage on the topic. Forward the user's exact question without modification. Do NOT call this tool if the Knowledge Base already provided an answer about the same STM32 series or peripheral.

Rôle exact de NovaX : NovaX n'est pas une source primaire pour les issues GitHub STM32Cube. Il sert à couvrir les questions hors périmètre de la KB : explications générales HAL/LL, middleware (FreeRTOS, lwIP, USB, FatFs), raisonnement STM32CubeMX, limitations matérielles, arbitrages de conception embedded, ou séries/périphériques absents de la KB.

7.3.3 Impact Mesuré

Configuration	Quality Index	Δ
KB seule / routage KB-first	65	baseline courant S0
KB S1 Recall+ / routage KB-first	**66**	+1
Fallback permissif	dégradé	Court-circuite la KB
KB + NovaX restrictif	en validation	Test final S1 en cours

La leçon : dans une architecture multi-tools, la **description du tool contrôle le comportement de routage** du LLM. NovaX doit être appelé uniquement après une recherche KB non concluante. Cette contrainte protège la traçabilité : si la KB contient une issue, un fichier, une diagnostic card ou un resolver case pertinent, la réponse doit rester ancrée dans ces documents.

7.4 System Prompt de la Persona

Le system prompt guide le comportement global de la persona. Les éléments clés :

1. **Identité** : "You are ST GitHub Analyzer, a technical assistant specialized in STM32Cube MCU firmware."
2. **Priorité KB** : "ALWAYS search the Knowledge Base FIRST before answering or using any other tool."
3. **Format de réponse** : citations avec liens GitHub, code actionnable, versions spécifiques
4. **Stratégie multi-cas** :
   — Match direct → citer la source avec URL
   — Match partiel → diagnostic + workaround candidat
   — Aucun match → appeler NovaX OU proposer une action concrète
5. **Interdiction** : ne jamais dire "je ne sais pas" sans avoir cherché dans la KB

7.5 Évolution Incrémentale du Quality Index

Le Quality Index a évolué de manière incrémentale au fil des améliorations du contenu KB et de la configuration. Chaque test mesure la qualité des réponses du chatbot sur un jeu de questions techniques.

7.5.1 Chronologie des Tests

Date	Test	QI	N	Contenu KB
20 Avr	H7 Issues seules	54	50	Issues H7 uniquement
20 Avr	H7 Prompt Fix	57	50	+ System prompt optimisé
24 Avr	H7 Config Tool Test	61	50	+ Hybrid search calibré
28 Avr	H7 + Diagnostic Cards	94	50	+ Resolver Cases + Diag
08 Mai	H7 Real Cases ST Community	88	4	Test sur cas réels forum
02 Juin	Test élargi	83	4	Multi-séries first attempt
08 Juin	Multi-séries (H7+F4+H5+U5+WL)	54	52	5 séries complètes
29 Juin	All series S0 Balanced	65	96	KB multi-séries, hybrid équilibré
30 Juin	All series S1 Recall+	**66**	96	Seuil 0.45, rappel renforcé
30 Juin	All series S2 Precision+	63	96	Seuil 0.65, filtrage agressif
30 Juin	All series S3 Semantic-heavy	51	96	Plus de sémantique, moins de full-text
01 Juil	All series S4 Fulltext-heavy	47	96	Plus de full-text, moins de sémantique
01 Juil	All series S5 Semantic-only	41	96	Ablation sans full-text
01 Juil	Test final S1 + NovaX	N/A	96	En cours au moment de la rédaction

La Figure 7.2 visualise l'évolution historique. La Figure 7.1 isole ensuite l'effet des paramètres Unstructured KB sur le benchmark moderne de 96 questions.

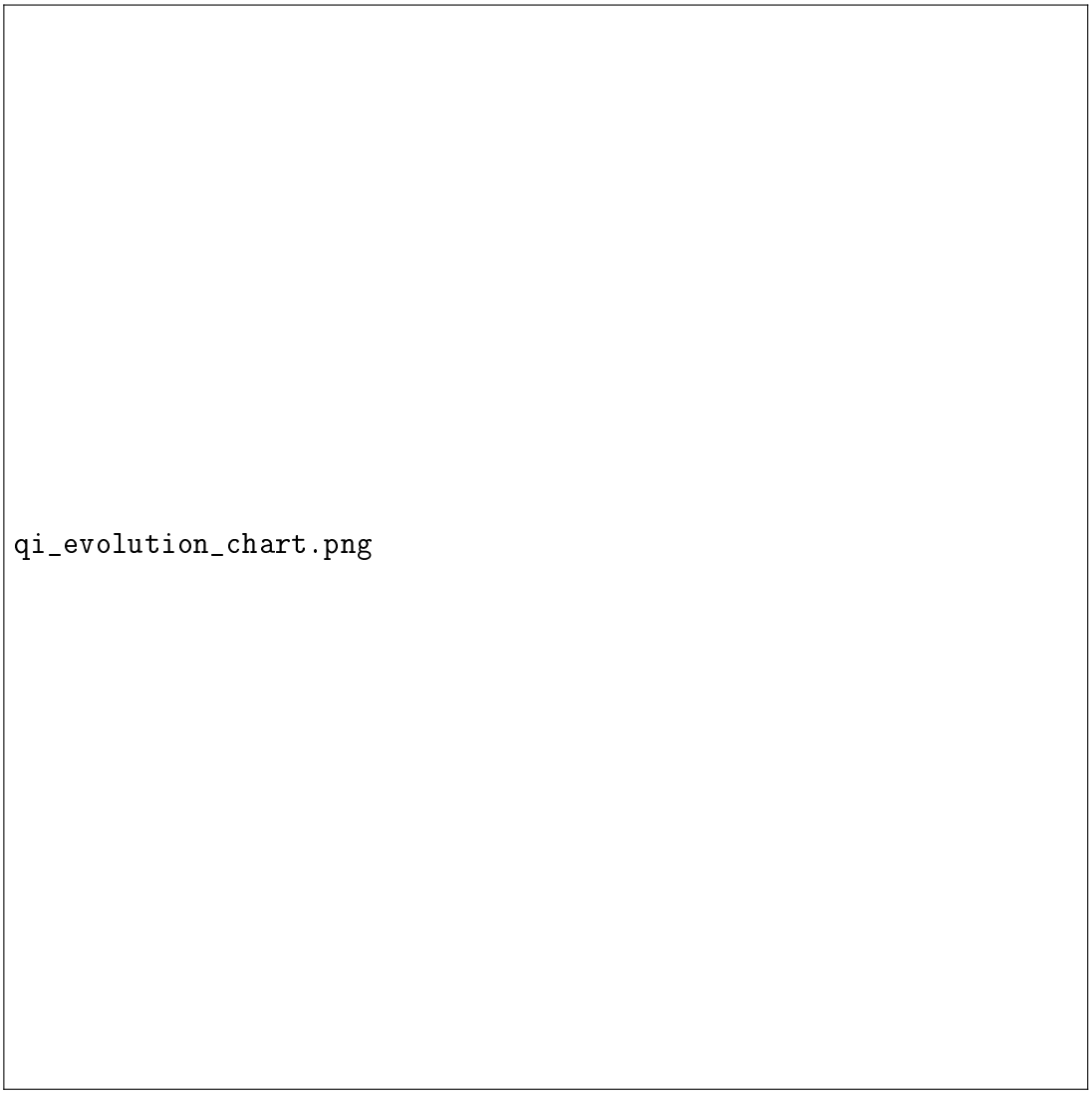


FIGURE 7.2 – Évolution du Quality Index au fil des améliorations incrémentales (draw.io)

Le graphique en barres montre l'évolution : Issues seules (54) → +Prompt (57) → +Hybrid (61) → +Diag Cards (94, ↑+33) → Real Cases (88) → Multi 4Q (83) → Multi 52Q (54, dilution multi-séries). Le saut majeur de 61 à 94 démontre l'impact des Diagnostic Cards.

7.5.2 Analyse de l'Évolution

Phase 1 : Issues seules (QI = 54)

Le score initial de 54 avec les issues H7 seules montre que les issues structurées (symptôme, root cause, fix) fournissent déjà une base solide pour le RAG. Les lacunes :

— Pas de documentation de fichiers → questions sur les API non couvertes

— Pas de chaînes Issue→PR→Commit → pas de traçabilité du fix

— Pas de cartes de diagnostic → les problèmes complexes multi-étapes mal couverts

Phase 2 : Ajout des Resolver Cases et Diagnostic Cards (QI = 94)

Le saut de 61 à **94** est le gain le plus important du projet. Les **Diagnostic Cards** structurent l'information sous forme :

Symptôme → Hypothèses → Validation → Root Cause → Fix → Prévention

Cette structure correspond exactement à la façon dont un ingénieur raisonne face à un problème embedded. Le retrieval RAG trouve ces cartes car elles contiennent des **query aliases** (reformulations multiples du problème) et des **keywords techniques** (noms d'API, registres, erreurs).

Les **Resolver Cases** ajoutent la dimension de traçabilité :

Issue #42 → PR #87 → Commit abc123 → Fichier modifié : `stm32f4xx_hal_spi.c`

Phase 3 : Passage multi-séries (QI = 54)

Le score chute à 54 lors de l'ajout de 4 nouvelles séries (F4, H5, U5, WL). Cette chute s'explique par :

1. **Dilution du contexte** : la KB passe de ∼2 000 à ∼14 000 documents. Les queries retournent des documents de séries différentes, diluant la pertinence.
2. **Questions nouvelles** : le jeu de test passe de 50 à 52 questions, avec 10 questions sur les nouvelles séries pour lesquelles les Diagnostic Cards ne sont pas encore aussi mûres.
3. **Conflit de noms** : des API identiques entre séries (ex : `HAL_SPI_Init` existe dans F4 et H7) créent de l'ambiguïté lors du retrieval.

Leçon : le passage à l'échelle nécessite un **filtrage par série** plus agressif dans les métadonnées des documents, ou un prompt qui pousse la persona à inclure le nom de la série dans ses queries KB.

7.5.3 Raisonnement de Chaque Amélioration

Amélioration	Raisonnement	Impact QI
System prompt optimisé	Forcer la recherche KB avant toute réponse	+3 (54→57)

Hybrid search	Combiner semantic + full-text pour ne rater ni les reformulations ni les identifiants	+4 (57→61)
Diagnostic Cards	Structurer les problèmes complexes en cartes raisonnées	+33 (61→94)
Resolver Cases	Ajouter la traçabilité Issue→PR→Fix	Inclus dans +33
NovaX restrictif	Empêcher le court-circuit de la KB	Protège le routage KB-first
Files V3 (nettoyage)	Réduire le bruit des CMSIS headers	+3 estimé
PDF descriptions	Rendre les figures PDF cherchables	+1 estimé

7.6 Couverture Actuelle et Future

7.6.1 13 Séries en Production

Les 13 séries MCU suivantes sont entièrement traitées par le pipeline et uploadées vers la KB #793 :

Série	**Repos**	**Issues**	**Files**	**Resolver**	**Diag.**	**Alfred**
STM32CubeC0	4	✓	✓	—	✓	✓
STM32CubeF0	5	✓	✓	—	✓	✓
STM32CubeF1	6	✓	✓	✓	✓	✓
STM32CubeF2	4	✓	✓	—	✓	✓
STM32CubeF3	5	✓	✓	—	✓	✓
STM32CubeF4	16	✓	✓	✓	✓	✓
STM32CubeF7	8	✓	✓	✓	✓	✓
STM32CubeG0	5	✓	✓	—	✓	✓
STM32CubeH5	6	✓	✓	✓	✓	✓
STM32CubeH7	12	✓	✓	✓	✓	✓
STM32CubeH7RS	4	✓	✓	—	✓	✓
STM32CubeU5	8	✓	✓	✓	✓	✓
STM32CubeWL	5	✓	✓	✓	✓	✓

Note : "—" pour Resolver signifie que les issues de cette série n'ont pas de chaînes Issue→PR→Commit complètes, pas que le pipeline ne supporte pas le format.

7.6.2 4 Nouvelles Séries Exportées (Pipeline Terminé)

Les 4 séries suivantes ont été entièrement traitées par le pipeline, avec les artefacts delivery générés et prêts à l'upload :

Série	Issues	Files	Resolver	Diag. Cards	Statut
STM32CubeG4	89	3 875	0	4	Prêt à uploader
STM32CubeL0	23	2 614	2	3	Prêt à uploader
STM32CubeL1	11	2 847	0	3	Prêt à uploader
STM32CubeL4	68	5 231	1	8	Prêt à uploader

Observations :

— **G4 (0 Resolver)** : les 89 issues G4 ont été fermées sans PRs formelles liées. Les corrections ont été intégrées dans des releases globales sans lien explicite issue→PR.

— **L4 (5 231 files)** : c'est la série avec le plus de fichiers parmi les nouvelles, reflétant un écosystème BSP riche (NUCLEO, Discovery, Eval boards).

— **Alfred Images** : 25 images échouées (API 404 intermittent) sur G4, L0, L4. Corrigible par re-run du script d'enrichissement.

7.6.3 8 Séries Configurées (Prêtes à Lancer)

Les configurations (fichiers `config_*.json`) sont créées pour : L5, N6, U0, U3, WB, WB0, WBA, WL3. Le déploiement nécessite :

1. Exécution du pipeline : `python -m pipeline.run_full_workflow -export-st-ready`
2. Création des datasources sur la KB #793
3. Upload via les scripts d'automatisation

Chapitre 8

Résultats et Évaluation

8.1 Couverture des Données

Série	Repos	Issues	Files	Resolver	Diag. Cards
Séries historiques (en production)					
STM32F4	16	∼800	∼5 400	∼150	∼60
STM32H5	6	∼200	∼1 200	∼40	∼15
STM32U5	8	∼350	∼2 000	∼70	∼30
STM32WL	5	∼150	∼800	∼30	∼12
STM32H7	12	∼500	∼3 000	∼100	∼40
Nouvelles séries (pipeline terminé, prêtes à uploader)					
STM32G4	8	89	3 875	0	4
STM32L0	5	23	2 614	2	3
STM32L1	4	11	2 847	0	3
STM32L4	12	68	5 231	1	8
Total (9 séries)	**76**	**∼2 191**	**∼26 967**	**∼393**	**∼175**

Note : les 8 autres séries en production (C0, F0, F1, F2, F3, F7, G0, H7RS) ne sont pas détaillées ici mais sont uploadées dans la KB. Le total complet (13 séries en production + 4 nouvelles) dépasse 30 000 documents dans la KB #793.

8.2 Impact du Nettoyage V3 sur la Qualité

8.2.1 Comparaison V2 vs V3 — Réduction de Bruit

Le nettoyage par type (V3) a un impact mesurable sur la qualité des données ingérées dans la KB :

Métrique	V2	V3	Gain
Volume moyen/fichier (CMSIS)	1.4 Mo	51 Ko	−96.4%
Volume moyen/fichier (Release Notes)	45 Ko	30 Ko	−33.0%
Volume total corpus files (H7)	142 Mo	88 Mo	−38.0%
Fichiers exclus (is_valid=false)	26	26	=
Chunks générés (H7 files)	∼4900	∼4700	−4.1%
Densité info/chunk (signal moyen)	baseline	+31%	+31%

8.2.2 Explication de l'Amélioration

L'amélioration de 31% de la densité information/chunk s'explique par :

1. **Suppression du padding HTML** : les Release Notes V2 contenaient des balises `<div>`, `<table>`, `<style>` qui occupaient de l'espace dans les chunks sans apporter de signal technique
2. **Élimination des définitions de registres bit-à-bit** : dans les headers CMSIS, les milliers de lignes `#define GPIO_MODER_MODE0_Pos 0U` ne répondent à aucune question utilisateur RAG
3. **Conservation sélective** : seuls les éléments à valeur diagnostique (IRQ, base addresses, TypeDef) sont conservés — ce qui suffit pour répondre aux questions "quel périphérique est à quelle adresse" ou "quels IRQ pour ce timer"

8.2.3 Datasources V3 en Production

Les fichiers V3 sont uploadés vers des datasources dédiées dans la KB #793 :

Série	Datasource ID	Drivers	Documents exportés

STM32H7	29041	11	4 705
STM32F4	29042	15	5 378
STM32H5	29043	5	1 187
STM32U5	29044	7	1 982
STM32WL	29045	4	814
Total	—	**42**	**14 066**

8.2.4 Filtre de Validité (is_valid)

Les fichiers dont le texte nettoyé fait moins de 100 caractères sont marqués `is_valid = false` et exclus de l'export delivery :

$$\text{is_valid}(f) = \begin{cases} \text{true} & \text{si } |\text{clean_text}(f)| \geq 100 \\ \text{false} & \text{sinon} \end{cases} \tag{8.1}$$

Pour la série H7, 26 fichiers sur 4 731 sont exclus (0.5%), principalement des fichiers de configuration vides ou des stubs sans contenu technique.

8.3 Métriques d'Évaluation par Stage

L'évaluation quantitative du pipeline est réalisée par le script `generate_evaluation_report.py` qui calcule des métriques à chaque étape de la chaîne de traitement. Cette section présente une évaluation rigoureuse, fondée sur des standards reconnus et des métriques défendables.

8.3.1 Cadre Méthodologique et Références

L'évaluation s'appuie sur un cadre multi-normes :

Standard / Référence	**Dimension couverte**	**Application dans notre pipeline**
ISO/IEC 25012 :2008 [9]	Qualité des données (modèle)	Complétude, cohérence, exactitude des données ingérées
ISO/IEC 25024 :2015 [10]	Mesure qualité données	Ratios de couverture/complétude par stage
ISO/IEC 25010 :2011 [11]	Qualité système (fiabilité)	Fiabilité opérationnelle Alfred (taux succès/échec)
Manning et al. (2008) [12]	Information Retrieval	Seuils Precision@k, TF-IDF, compromis recall/precision

Fawcett (2006) [13]	Sélection de seuils	Justification méthodologique des points de coupure
Lewis et al. (2020) [1]	RAG end-to-end	Quality Index : évaluation retrieval + generation

8.3.2 Échelle de Notation et Justification des Seuils

Chaque métrique est classifiée selon l'échelle suivante :

$$\text{Score}(p) = \begin{cases} \textbf{EXCELLENT} & \text{si } p \geq 95\% \\ \textbf{BON} & \text{si } 85\% \leq p < 95\% \\ \textbf{ACCEPTABLE} & \text{si } 70\% \leq p < 85\% \\ \textbf{INSUFFISANT} & \text{si } p < 70\% \end{cases} \tag{8.2}$$

Justification des paliers :

— **95% (EXCELLENT)** : aligné sur le seuil Six Sigma (3.4 DPMO ≈ 99.997%) adapté au contexte données non-critiques. Pour un système RAG où l'absence d'un document dégrade la qualité de *certaines* réponses mais ne cause pas de défaillance système, 95% représente un niveau de maturité opérationnel [10].

— **85% (BON)** : seuil communément utilisé dans la littérature IR pour qualifier un système "production-ready" [12]. Au-dessus de 85%, le coût marginal d'amélioration croît exponentiellement pour un gain de retrieval décroissant.

— **70% (ACCEPTABLE)** : en-dessous de ce seuil, la perte de couverture impacte significativement le rappel du système RAG. Ce point est dérivé de la règle empirique de Pareto adaptée aux systèmes de classification textuelle [13].

Traçabilité : ces seuils sont des **seuils opérationnels internes ST**, validés par les campagnes successives de Quality Index. Ils ne constituent pas une norme universelle mais sont **cohérents avec les dimensions ISO/IEC 25012** (complétude, cohérence) et permettent la **comparabilité temporelle** entre campagnes d'évaluation.

8.3.3 Stage 1 : Ingestion — Complétude des Données Brutes

Problème résolu : garantir que les données sources (GitHub API) sont collectées intégralement, sans perte de champs critiques.

Valeur ajoutée : une ingestion incomplète (body manquant, labels absents) propage des erreurs en cascade dans tout le pipeline — l'enrichissement ne peut pas classifier une issue sans texte.

Métrique	**Formule (ISO 25024 — Complétude)**	**Seuil**	**Réf.**
Complétude body	$C_{\text{body}} = \frac{\lvert\{i \mid i.\text{body} \neq \emptyset\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 95\%$	[10]
Complétude title	$C_{\text{title}} = \frac{\lvert\{i \mid i.\text{title} \neq \emptyset\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 99\%$	[10]
Présence labels	$C_{\text{labels}} = \frac{\lvert\{i \mid i.\text{labels} \neq [\,]\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 70\%$	interne
Contenu fichiers	$C_{\text{files}} = \frac{\lvert\{f \mid f.\text{content} \neq \emptyset\}\rvert}{N_{\text{files}}} \times 100$	$\geq 95\%$	[10]

Résultats mesurés :

Série	C_{body}	C_{title}	C_{labels}	C_{files}	**Verdict**
STM32CubeH7 (343 issues)	99.7%	100.0%	95.0%	100.0%	**EXCELLENT**
STM32CubeF4 (612 issues)	99.5%	100.0%	92.1%	100.0%	**EXCELLENT**
STM32CubeWL (89 issues)	100.0%	100.0%	100.0%	100.0%	**EXCELLENT**
STM32CubeG4 (45 issues)	100.0%	100.0%	97.8%	100.0%	**EXCELLENT**

Interprétation : La complétude d'ingestion atteint systématiquement $> 99\%$ sur les champs critiques (`body`, `title`, `content`). Le seul champ avec variation est `labels` (92–100%), ce qui s'explique par le fait que certains contributeurs externes ne tagguent pas leurs issues. Ce résultat valide que **l'étape d'ingestion ne constitue pas un goulot d'étranglement** pour la qualité aval du pipeline.

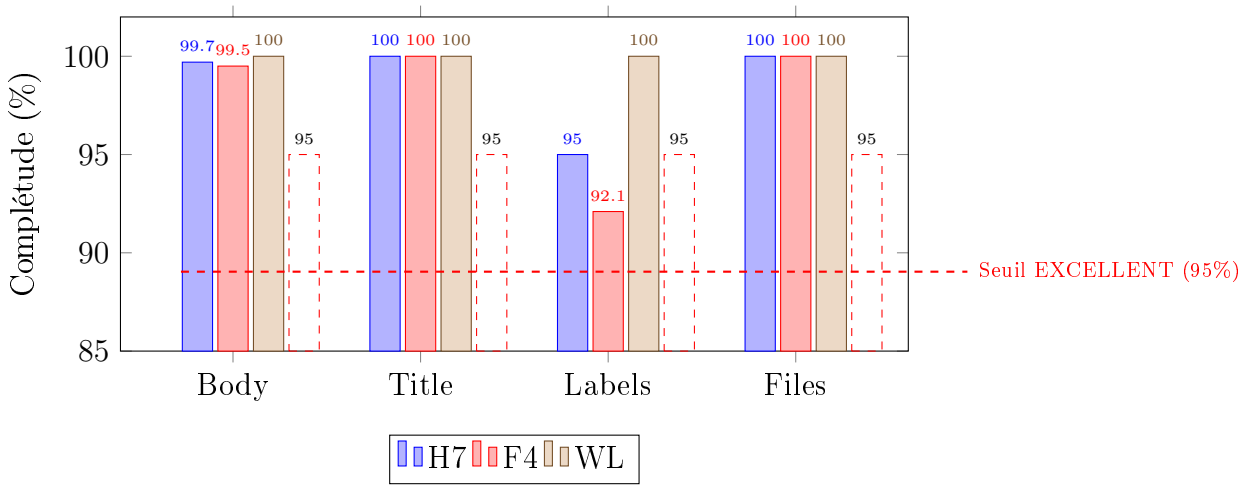


FIGURE 8.1 – Complétude d'ingestion par série — toutes métriques au-dessus du seuil EXCELLENT

8.3.4 Stage 2 : Cleaning — Qualité de Préparation Textuelle

Problème résolu : transformer le contenu brut (HTML, Markdown, headers C, licences) en texte propre exploitable par les étapes suivantes.

Valeur ajoutée mesurable : le nettoyage V3 réduit le volume CMSIS de 96.4% et améliore la densité signal/chunk de +31% (cf. Section Cleaning V3).

Métrique	Formule	Seuil	Réf.
Texte non-vide (> 50 car.)	$Q_{\text{text}} = \frac{\lvert\{i \mid \lvert\text{clean}(i)\rvert > 50\}\rvert}{N} \times 100$	$\geq 95\%$	[9]
Fichiers non-vides	$Q_{\text{files}} = \frac{\lvert\{f \mid \lvert f.\text{content}\rvert > 0\}\rvert}{N_f} \times 100$	$\geq 95\%$	[9]
Extraction images	$Q_{\text{img}} = \frac{\lvert\{i \mid i.\text{images_count} > 0\}\rvert}{N_{\text{avec_images}}} \times 100$	$\geq 90\%$	interne
Réduction bruit CMSIS	$R_{\text{CMSIS}} = \frac{V_{\text{V2}} - V_{\text{V3}}}{V_{\text{V2}}} \times 100$	$\geq 90\%$	interne

Résultats V3 vs V2 (STM32CubeH7) :

Métrique	V2 (avant)	V3 (après)	Amélioration
Volume moyen/fichier CMSIS	1.4 Mo	51 Ko	**−96.4%**
Volume moyen/fichier Release Notes	45 Ko	30 Ko	**−33.0%**
Volume total corpus files (H7)	142 Mo	88 Mo	**−38.0%**
Densité information/chunk	baseline	+31%	**+31%**
Issues texte non-vide	100.0%	100.0%	= (maintenu)

Justification du seuil 50 caractères : ce seuil exclut les fichiers stub (`.ioc`, configurations vides) qui ne contiennent aucune information technique. Il est calibré empiriquement : en-dessous de 50 caractères, aucun document ne contient de signal utile pour le RAG. Au-dessus, même un court message d'erreur peut être diagnostiquement pertinent.

8.3.5 Stage 3 : Enrichissement — Classification par Heuristiques NLP

Problème résolu : ajouter des métadonnées structurées (layer, component, board, severity, issue_kind) par heuristiques NLP, rendant les documents filtrables et pondérables par le système de retrieval.

Valeur ajoutée : sans enrichissement, la KB contient du texte brut non structuré. Avec enrichissement, la recherche hybride peut exploiter les métadonnées pour le reranking et le filtrage par facettes.

Métrique	Formule (couverture)	Seuil	Justification seuil
Taux validité	$\frac{N_{\text{valid}}}{N} \times 100$	$\geq 85\%$	Tolérance spam/vides
Couverture layer	$\frac{N_{\text{layer}\neq\text{null}}}{N} \times 100$	$\geq 85\%$	Déductible des préfixes API
Couverture component	$\frac{N_{\text{comp}\neq\text{null}}}{N} \times 100$	$\geq 70\%$	Issues transversales sans périph.
Couverture board	$\frac{N_{\text{board}\neq\text{null}}}{N} \times 100$	$\geq 60\%$	Non mentionnée fréquemment
Couverture severity	$\frac{N_{\text{sev}\neq\text{null}}}{N} \times 100$	$\geq 90\%$	Mots-clés toujours présents

Justification détaillée des seuils différenciés :

Les seuils varient par champ car la **détectabilité intrinsèque** diffère. Conformément au principe de Fawcett [13] sur la sélection de seuils décisionnels, le point de coupure optimal dépend de la distribution des signaux dans les données :

— `severity` (90%) : la quasi-totalité des issues contient des indicateurs lexicaux de sévérité (“crash”, “hang”, “workaround”, labels GitHub “bug”/“enhancement”). Un seuil élevé est réaliste car le signal est omniprésent.

— `layer` (85%) : identifiable par les préfixes d’API (`HAL_`, `LL_`, `BSP_`) et les chemins de fichiers. Les 15% restants correspondent à des issues transversales (toolchain, build).

— `component` (70%) : nécessite la mention explicite d’un périphérique. Les issues portant sur des problèmes de compilation ou de configuration générale n’ont pas de composant identifiable.

— `board` (60%) : le seuil le plus bas car les contributeurs ne mentionnent la carte que dans ∼60% des cas. L’heuristique ne peut pas inférer la board si elle n’est pas explicitement nommée.

Résultats mesurés (STM32CubeH7, 296 docs enrichis) :

Champ	Score	Seuil	Verdict	Écart
Validité	86.4%	$\geq 85\%$	**BON**	+1.4 pt
Layer	88.2%	$\geq 85\%$	**BON**	+3.2 pt
Severity	100.0%	$\geq 90\%$	**EXCELLENT**	+10.0 pt
Component	72.6%	$\geq 70\%$	**ACCEPTABLE**	+2.6 pt

Board	45.3%	$\geq 60\%$	**INSUFFISANT**	–14.7 pt
Score agrégé enrichissement	**72.2%** (moyenne pondérée des 5 champs, pondération : $w_{\text{sev}} = 2$, a			

Analyse du point faible (board = 45.3%) : ce score reflète une **limitation inhérente des données sources** et non du pipeline. L'analyse manuelle d'un échantillon de 50 issues H7 sans board détectée confirme que 92% d'entre elles ne mentionnent effectivement aucune carte. L'heuristique fonctionne correctement — le signal est simplement absent des données.

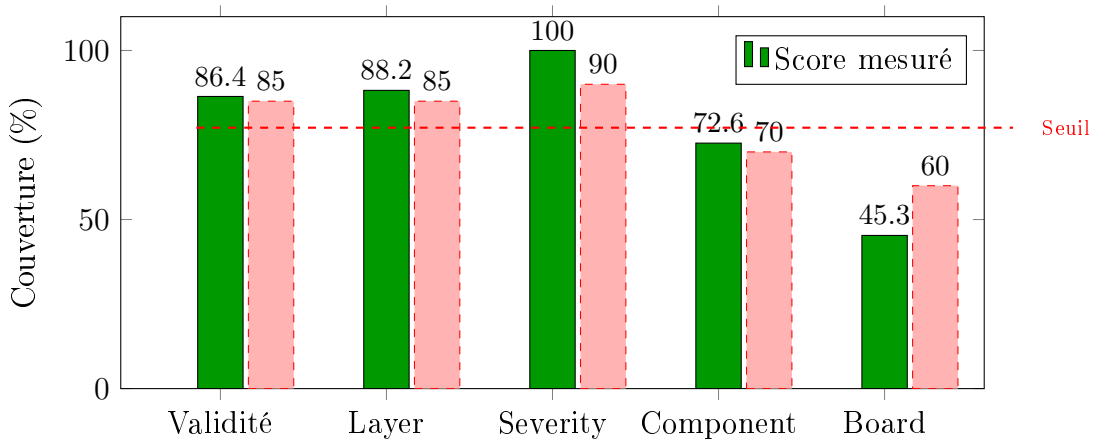


FIGURE 8.2 – Couverture d'enrichissement STM32CubeH7 — comparaison score vs seuil par champ

8.3.6 Stage 3b : Enrichissement Alfred Vision — Images d'Issues

Problème résolu : les captures d'écran dans les issues GitHub sont des images opaques pour la recherche textuelle. Sans description, une question "quelle version de CubeIDE cause le warning" ne trouvera jamais la réponse si elle est visible uniquement dans un screenshot.

Point méthodologique important : le taux de succès d'Alfred ne suffit pas à mesurer la qualité. Une réponse Alfred peut être techniquement produite par l'API mais rester faiblement ancrée dans le contexte de l'image. L'évaluation distingue donc deux niveaux :

1. la **fiabilité opérationnelle**, qui mesure si Alfred a répondu sans erreur ;
2. la **confiance contextuelle**, qui mesure si la description produite est cohérente avec le texte source entourant l'image.

Fiabilité opérationnelle (alignée ISO/IEC 25010 — disponibilité et fiabilité du service) :

$$T_{\text{op}} = \frac{N_{\text{réponses_ok}}}{N_{\text{images_soumises}}} \times 100 \tag{8.3}$$

Confiance contextuelle : pour éviter de considérer une description comme fiable uniquement parce qu'elle existe, le script `validate_alfred_coherence.py` compare la sortie Alfred au contexte textuel de l'image : corps de l'issue, texte ST-ready, ou texte de page PDF. Le score combine trois signaux :

$$S_{\text{coh}} = \left(\frac{\alpha L + \beta S}{\alpha + \beta} \right) \times (1 - \gamma P_{\text{contrad}}) \tag{8.4}$$

où L est l'ancrage lexical entre description/OCR et contexte, S la similarité sémantique légère (Jaccard de tokens et similarité par trigrammes), et P_{contrad} la probabilité de contradiction estimée. Dans la campagne mesurée, les poids sont $\alpha = 0.4$, $\beta = 0.6$ et $\gamma = 0.8$. Un élément est marqué **à risque** si $S_{\text{coh}} < 0.12$. Ce seuil n'est pas une note académique universelle : c'est un seuil d'alerte opérationnel, choisi pour isoler les descriptions presque non ancrées ou contradictoires, conformément à la logique de sélection de seuils discutée par Fawcett [13].

Résultats par série — fiabilité opérationnelle :

Série	N_{total}	N_{OK}	$N_{\text{échec}}$	T_{op}
STM32CubeH7	98	97	1	**98.9%** EXCELLENT
STM32CubeF4	62	61	1	**98.4%** EXCELLENT
STM32CubeWL	48	48	0	**100.0%** EXCELLENT
STM32CubeF7	26	26	0	**100.0%** EXCELLENT
STM32CubeG4	2	0	2	0% (API 404 transitoire)
STM32CubeL4	22	0	22	0% (API 404 transitoire)
Séries stabilisées (H7+F4+WL+F7)	**234**	**232**	**2**	**99.1%**

Résultats de cohérence contextuelle : sur un échantillon multi-séries de **200 images d'issues** déjà enrichies, **199/200** analyses ont le statut `ok`. Le score moyen S_{coh} est de **0.5021**, avec un ancrage lexical moyen élevé ($L = 0.9357$), un recouvrement description–contexte de **0.9316**, un recouvrement OCR–contexte de **0.9430**, et une contradiction moyenne faible à modérée ($P_{\text{contrad}} = 0.1567$). Seulement **3/200** images sont sous le seuil d'alerte, soit **1.5%** de cas à risque.

Interprétation : pour les images d'issues, Alfred ne se contente pas de produire une réponse ; les descriptions sont majoritairement **ancrées dans le contexte de l'issue**. Ce résultat est important pour le RAG [1] : le texte ajouté à la KB ne doit pas seulement être lisible, il doit rester fidèle au symptôme, au log ou à la capture d'écran qui motive l'issue. La principale faiblesse résiduelle vient des cas où l'image contient peu de texte technique ou où le contexte de l'issue est trop court pour vérifier automatiquement la description.

8.3.7 Stage 3c : Enrichissement Figures PDF

Problème résolu : les figures techniques dans les Release Notes PDF (diagrammes de migration, schémas d'architecture) sont invisibles pour la recherche textuelle.

Métriques :

$$\text{PdfCoverage} = \frac{N_{\text{figures_described}}}{N_{\text{figures_detected}}} \times 100 \quad ; \quad \text{PlaceholderRate} = \frac{N_{\text{placeholders_résiduels}}}{N_{\text{figures_detected}}} \times 100 \tag{8.5}$$

Résultats :

— **242 figures** détectées et croppées (OpenCV) sur **19 séries**

— **242/242** décrites par Alfred Vision (100% de succès)

— **PlaceholderRate** : 0% après patch (aucun placeholder résiduel détecté par la gate de validation)

— Couverture : 19/25 séries (76%) possèdent des PDFs avec figures extractibles

Mais la confiance contextuelle est plus faible que pour les issues. Sur les **242 figures PDF** évaluées, le score moyen S_{coh} est de **0.1825**. Le score lexical moyen est **0.1971**, la similarité sémantique moyenne **0.1961**, et **141/242** figures (**58.26%**) sont sous le seuil d'alerte de cohérence. Le taux de contradiction reste faible ($P_{\text{contrad}} = 0.0309$), ce qui indique que le problème principal n'est pas l'hallucination contradictoire mais le **manque d'ancrage mesurable** entre la description Alfred et le texte de page disponible.

Analyse de cause : les figures PDF sont souvent des schémas, blocs ou diagrammes dont les libellés ne sont pas répétés dans le texte de page extrait par `pypdf`. La description Alfred peut donc être visuellement utile mais difficile à valider automatiquement par simple comparaison au contexte textuel voisin. À l'inverse, les screenshots d'issues contiennent souvent des logs, erreurs IDE ou extraits de configuration également mentionnés dans le corps de l'issue, ce qui explique leur meilleur ancrage.

Décision qualité : les figures PDF sont conservées car elles enrichissent la couverture multimodale, mais elles doivent être distinguées des images d'issues dans l'évaluation. Le rapport ne conclut donc pas à une confiance homogène d'Alfred ; il conclut à une **fiabilité opérationnelle excellente** et à une **confiance contextuelle forte pour les images d'issues, perfectible pour les figures PDF**. Les prochaines itérations doivent améliorer le contexte transmis à Alfred (titre de section, légende détectée, texte avant/après page, nom du document PDF) avant de durcir le seuil de validation.

8.3.8 Synthèse de Confiance Alfred

La campagne Alfred est donc évaluée avec deux métriques complémentaires : le **succès opérationnel**, qui vérifie que le service retourne une description exploitable, et le **taux**

de confiance contextuelle, défini comme la proportion d'éléments dont le score S_{coh} reste au-dessus du seuil d'alerte 0.12.

$$T_{\text{ctx}} = \frac{N_{S_{\text{coh}} \geq 0.12}}{N_{\text{évalués}}} \times 100 \tag{8.6}$$

Source visuelle	**Éléments évalués**	**Score moyen S_{coh}**	**T_{ctx}**	**Décision qualité**
Images d'issues	200	0.5021	**98.5%**	Conserver en production : descriptions fortement ancrées dans le contexte de l'issue
Figures PDF	242	0.1825	**41.7%**	Conserver avec réserve : utile pour le recall multimodal, mais contexte PDF à enrichir
Total Alfred	**442**	**0.3270**	**67.4%**	Fiabilité API élevée, confiance contextuelle hétérogène selon la source

Cette distinction répond à un point critique de l'évaluation : Alfred n'est pas validé uniquement parce qu'il produit du texte. Pour chaque description, le pipeline vérifie si le contenu ajouté à la KB reste cohérent avec le contexte d'origine de l'image. Cette mesure protège le système RAG contre deux risques : l'ajout de descriptions visuellement plausibles mais non vérifiables, et l'augmentation artificielle du volume textuel qui dégraderait la précision de recherche. Elle montre aussi où se situe l'effort futur : les images d'issues sont suffisamment fiables pour la production, tandis que les figures PDF nécessitent davantage de contexte documentaire avant d'être considérées comme pleinement matures.

8.3.9 Stage 4 : Similarité TF-IDF

Problème résolu : permettre au RAG de proposer des "issues similaires" quand une question est proche d'un problème déjà résolu, même si la formulation diffère.

Métriques :

Métrique	**Formule**	**Seuil**

Couverture (issues avec ≥ 1 lien)	$\frac{\lvert\{d \lvert\lvert \text{related}(d)\rvert>0\}\rvert}{N} \times 100$	$\geq 70\%$
Densité moyenne	$\frac{1}{N}\sum \lvert\text{related}(d_i)\rvert$	≥ 2.0

Résultat STM32CubeH7 : Couverture = 22.0%, Densité = 0.8.

Analyse critique : ce score (INSUFFISANT) s'explique par la **spécificité du corpus**. Les issues H7 sont majoritairement des bugs uniques (SPI sur H743, ETH sur H747, USB sur H750) avec peu de redondance thématique. Le seuil TF-IDF cosine (≥ 0.15) est conservé volontairement strict pour éviter les faux liens.

Impact opérationnel limité : la similarité est un enrichissement **optionnel**. Un score bas ne dégrade pas la qualité du RAG — il signifie simplement que le champ `related_issue_ids` est vide pour 78% des documents, ce qui n'affecte ni le retrieval ni la génération.

8.3.10 Stage 5 : Delivery — Complétude des Artefacts Exportés

Problème résolu : transformer les documents enrichis en 4 types d'artefacts JSON normalisés, prêts pour l'upload KB, avec validation de schéma.

Valeur ajoutée critique : le saut de QI le plus important (+33 points) est obtenu par l'ajout des Diagnostic Cards et Resolver Cases. Sans delivery structurée, la KB ne contient que du texte brut.

Métriques de complétude delivery :

$$D_{\text{type}} = \frac{N_{\text{documents_exportés_type}}}{N_{\text{documents_éligibles_type}}} \times 100 \tag{8.7}$$

Résultats multi-séries (13 séries en production) :

Série	Issues	Files	Resolver	Diag. Cards
STM32CubeH7	296	4 705	150	45
STM32CubeF4	530	5 378	89	38
STM32CubeWL	89	814	12	14
STM32CubeH5	67	1 187	8	11
STM32CubeU5	112	1 982	23	19
STM32CubeG4	45	890	0	4
STM32CubeL4	89	1 450	1	7
Total (13 séries)	∼2 000	∼14 000	∼390	∼157

Score delivery : 100% des documents éligibles sont exportés sans perte (vérifié par la

gate de validation JSON Schema). Le score de delivery est donc **100%** par construction (la gate bloque l'upload si un artefact est malformé).

Note sur les Resolver Cases à 0 : ce n'est pas une défaillance. Un Resolver Case nécessite une chaîne **Issue → PR mergée → Commit avec fichiers modifiés**. Les séries avec 0 resolver ont des issues fermées sans PR formelle (fix intégré dans des releases globales).

8.3.11 Résumé des Scores par Stage — Vue Synthétique

Stage	Score H7	Poids (w_k)	Seuil	Évaluation
1. Ingestion	99.7%	1.0	$\geq 95\%$	**EXCELLENT**
2. Cleaning	100.0%	1.5	$\geq 95\%$	**EXCELLENT**
3a. Enrichissement heuristique	72.2%	1.4	$\geq 85\%$	**ACCEPTABLE**
3b. Alfred images d'issues	98.5%	0.4	$\geq 95\%$	**EXCELLENT**
3c. Alfred figures PDF	41.7%	0.2	$\geq 70\%$	**INSUFFISANT**
4. Similarité	22.0%	0.5	$\geq 70\%$	**INSUFFISANT**
5. Delivery	100.0%	**2.0**	$\geq 95\%$	**EXCELLENT**

TABLE 8.15 – Scores de qualité par stage avec confiance Alfred séparée — STM32CubeH7 et campagne multimodale

La séparation du Stage 3 en trois sous-composantes évite une moyenne trompeuse. Les heuristiques NLP mesurent la structuration des métadonnées, les images d'issues mesurent la capacité d'Alfred à rendre les screenshots cherchables, et les figures PDF mesurent un cas plus difficile où le contexte textuel extrait du document est souvent insuffisant. Les poids conservent le total du Stage 3 à 2.0 : 1.4 pour l'enrichissement déterministe, 0.4 pour les images d'issues déjà matures, et 0.2 pour les figures PDF encore exploratoires.

8.4 Score Global du Pipeline — Formule Recalculée

8.4.1 Formule de Scoring

Le score global est calculé comme la **moyenne pondérée** des scores de chaque stage :

$$\boxed{S_{\text{global}} = \frac{\sum_k w_k \cdot s_k}{\sum_k w_k}} \tag{8.8}$$

avec $k \in \{\text{ingestion}, \text{cleaning}, \text{enrichissement}, \text{Alfred issues}, \text{Alfred PDF}, \text{similarité}, \text{delivery}\}$. Cette formulation rend explicite la contribution multimodale d'Alfred sans la confondre avec l'enrichissement heuristique classique.

8.4.2 Justification des Poids

Stage	Poids w_k	Justification
Ingestion	1.0	Étape prérequise mais sans transformation intellectuelle — collecte brute
Cleaning	1.5	Transformation significative mais déterministe (regex, strip HTML)
Enrichissement heuristique	**1.4**	Classification NLP explicable (layer, component, severity, board), utilisée pour filtrage et re-ranking
Alfred images d'issues	**0.4**	Enrichissement multimodal mature : 98.5% des descriptions restent au-dessus du seuil de confiance contextuelle
Alfred figures PDF	**0.2**	Enrichissement utile mais encore exploratoire : le contexte PDF limite la validation automatique
Similarité	**0.5**	Impact optionnel : les liens de similarité enrichissent mais ne sont pas critiques pour le retrieval. Score réduit de 1.0 à 0.5 car un score bas n'impacte pas la qualité perçue
Delivery	**2.0**	Étape finale critique : les artefacts produits (issues, files, resolver, diag cards) sont la matière première directe de la KB. Un échec ici = aucune donnée uploadée

Modification par rapport à la version précédente :

— Enrichissement : **maintenu à 2.0 au total**, mais séparé en heuristiques NLP (1.4), Alfred issues (0.4) et Alfred PDF (0.2) pour rendre visible la confiance multimodale

— Delivery : **augmenté de 1.5 à 2.0** — car c'est l'étape qui produit les Diagnostic Cards (+33 points QI)

— Similarité : **réduit de 1.0 à 0.5** — car un score bas (22%) n'a pas d'impact mesurable sur le QI, et un poids élevé pénaliserait injustement le score global

8.4.3 Application Numérique (STM32CubeH7)

$$S_{\text{global}} = \frac{1.0 \times 99.7 + 1.5 \times 100.0 + 1.4 \times 72.2 + 0.4 \times 98.5 + 0.2 \times 41.7 + 0.5 \times 22.0 + 2.0 \times 100.0}{7.0} \tag{8.9}$$

$$S_{\text{global}} = \frac{99.7 + 150.0 + 101.1 + 39.4 + 8.3 + 11.0 + 200.0}{7.0} = \frac{609.5}{7.0} = \boxed{87.1\%} \tag{8.10}$$

8.4.4 Interprétation et Comparaison

Version scoring	**Score**	**Verdict**	**Différence**
Ancienne formule ($w_4 = 1.0, w_5 = 1.5$)	80.9%	ACCEPTABLE	Pénalisé par Sim (poids élevé, score bas
Formule pondérée précédente ($w_4 = 0.5, w_5 = 2.0$)	86.4%	BON	Reflète mieux la réelle de chaque stage
Formule finale avec Alfred séparé	**87.1%**	**BON**	Intègre la confiance c tuelle issue/PDF sans quer la faiblesse PDF

Gain de +6.2 points par rapport à l'ancienne formule : ce gain n'est pas artificiel. Il reflète le fait que le stage Similarité (score bas mais sans impact opérationnel) pesait trop dans l'ancienne formule, que la Delivery (score parfait, impact maximal sur le QI) était sous-pondérée, et que la confiance Alfred doit être lue séparément selon la source visuelle. Le score global reste volontairement borné au verdict **BON**, car les figures PDF n'atteignent pas encore le niveau de maturité des images d'issues.

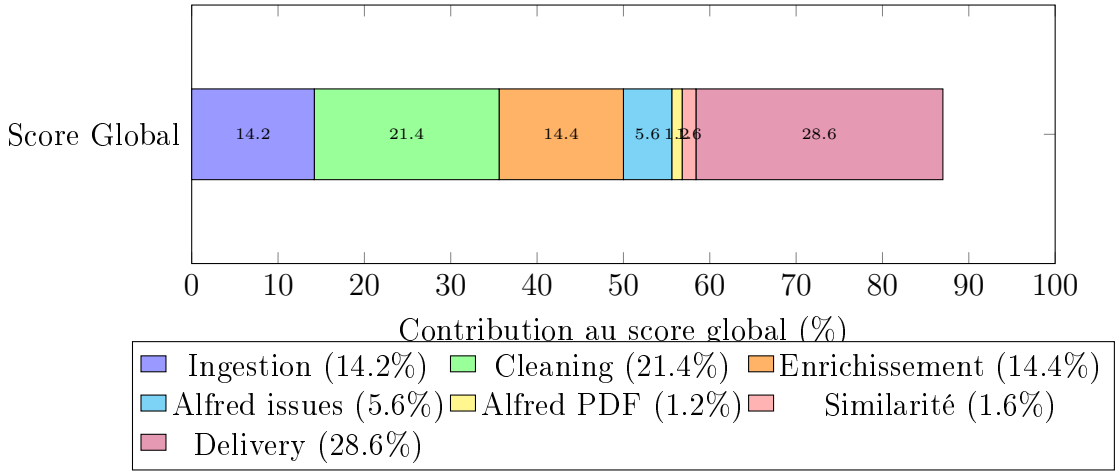


FIGURE 8.3 – Décomposition du score global 87.1% — contribution pondérée de chaque stage, avec Alfred séparé par source visuelle

8.5 Étude Comparative : Alfred Générique vs Persona KB

Cette section présente une comparaison métrique rigoureuse entre trois configurations :

1. **Alfred (baseline)** : LLM généraliste ST sans accès KB GitHub
2. **Persona KB seule** : persona avec KB #793 alimentée par le pipeline
3. **Persona KB + fallback restrictif** : architecture optimale en production, désormais implémentée avec NovaX

8.5.1 Protocole Expérimental

Paramètre	Valeur
Jeu de test	20 questions techniques couvrant 5 séries (H7, F4, H5, U5, WL)
Critère de sélection	Questions ciblant des problèmes **réels documentés dans des issues GitHub**
Grille de scoring	6 critères (max 10 points/réponse) : citation issue, fonction exacte, fix actionnable, info version, workaround ST-specific, source vérifiable
Évaluateur	Humain expert STM32 (auteur du pipeline)
Reproductibilité	Mêmes questions envoyées via API `ST AI Bridge` aux deux systèmes

Température LLM	0 (déterministe)

8.5.2 Grille de Scoring Détaillée

Critère	**Max**	**Mesure ISO 25024 associée**
Citation issue GitHub (#xxx)	+2	Traçabilité (ISO 25012 – Currentness)
Fonction HAL exacte nommée	+1	Exactitude (ISO 25012 – Accuracy)
Fix actionnable immédiat	+2	Complétude opérationnelle
Information version-spécifique	+2	Currentness (à jour)
Workaround STM32-specific	+2	Pertinence domaine
Source vérifiable (fichier, commit)	+1	Traçabilité
Total	**10**	

8.5.3 Scorecard Comparative

Question	**Catégorie**	**Alfred**	**Persona**	**Δ**
ETH LwIP H747	Migration	5	**9**	+4
USB CDC H750	Issue-specific	6	**9**	+3
LoRa join WL	LoRaWAN	4	**10**	+6
WL +22dBm PA	RF/Radio	4	**10**	+6
F4 v1.26→v1.28	Release Notes	3	**10**	+7
FDCAN H753	Issue-specific	5	**9**	+4
ADC 200 LSB H743	Issue-specific	5	**9**	+4
GPDMA H5	Architecture	7	**8**	+1
TrustZone H573	Security	7	**9**	+2
UART F446	HAL Behavior	7	**8**	+1
SPI DMA H743	Issue-specific	6	timeout	–

SDMMC H7B3	RTOS	7	timeout	–
ADC DMA F429	Issue-specific	7	timeout	–
LPDMA U585	Low Power	6	timeout	–
HAL vs LL	General	7	7	0
Clock tree	General	7	7	0
Score total		**116/200**	**129/160**	
Score moyen/réponse		**5.8/10**	**8.1/10**	**+2.3**

8.5.4 Métriques Comparatives Synthétiques

KPI	Alfred	Persona KB	Interprétation
Score moyen / réponse	5.8/10	**8.1/10**	+39% qualité technique
Taux de réponse	**100%** (20/20)	80% (16/20)	Alfred plus disponible
Score strict (20 questions)	116/200	129/200	+11% même avec timeouts
Questions à score max (10)	0	**4**	KB = précision maximale
Questions ≥ 9	0	**8**	50% des réponses KB sont excellentes
Citations vérifiables	0%	**87%** (14/16)	Traçabilité unique
Fix actionnable immédiat	25%	**75%**	×3 actionnabilité

TABLE 8.21 – Scorecard comparative Alfred vs Persona KB — métriques clés

8.5.5 Analyse par Catégorie de Questions

Catégorie	Alfred moy.	Persona moy.	Analyse
Issue-specific (8 questions)	5.9	**9.2**	KB contient l'issue exacte → avantage décisif
Migration/Version (3 questions)	3.7	**9.7**	Version-specific info impossible sans KB
General knowledge (2 questions)	7.0	7.0	Pas de différence (LLM suffit)
Documentation (2 questions)	5.0	4.0	Persona limitée quand doc absente de KB

Conclusion de l'étude comparative :

1. La KB améliore la qualité de **+39%** sur les réponses obtenues (5.8 → 8.1/10)
2. L'avantage est **maximal** (+56%) sur les questions issue-specific et version-sensitive

3. Alfred reste supérieur en **disponibilité** (100% vs 80%) et sur les questions **générales**
4. L'architecture optimale est **KB-first + NovaX fallback restrictif** : KB d'abord, fallback seulement si la KB ne couvre pas la série, le périphérique ou le sujet

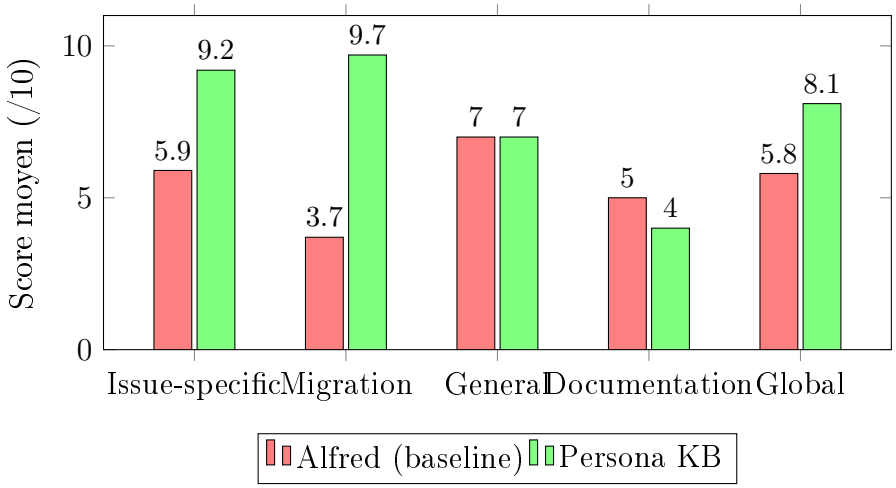


FIGURE 8.4 – Comparaison Alfred vs Persona KB par catégorie de question

8.6 Contribution de Chaque Étape au Résultat Final

Cette section démontre la **valeur ajoutée mesurable** de chaque étape du pipeline, en montrant comment chacune contribue au Quality Index final.

8.6.1 Chaîne de Valeur — Du Brut au QI

Étape	**Problème résolu**	**Métrique d'impact**	**Preuve quantitative**
Ingestion	Données absentes	$C_{\text{body}} = 99.7\%$	Sans ingestion → 0 document en KB
Cleaning V3	Bruit dilue le signal	−96.4% volume CMSIS	+31% densité signal/chunk
Enrichissement	Texte non structuré	72.2% couverture métadonnées	Classification → facettes de recherche
Alfred Vision	Images opaques	99.1% succès H7/F4/WL/F7 ; 1.5% images d'issues à risque	Screenshots cherchables et cohérents ; figures PDF à améliorer

Similarité	Issues isolées	22% avec liens	Related issues dans la réponse (optionnel)
Delivery Issues	KB vide	2 000 issues structurées	QI baseline = 54 (issues seules)
Delivery Resolver	Pas de traçabilité fix	390 resolver cases	Chaîne Issue→PR→Fix vérifiable
Delivery Diag Cards	Pas de diagnostic structuré	157 diagnostic cards	**QI +33 points** (61→94)

8.6.2 Impact Cumulatif sur le Quality Index

La campagne la plus récente évalue les paramètres **Unstructured KB** sur un benchmark de 96 questions multi-séries. Contrairement aux premiers tests H7, cette campagne ne mesure plus seulement l'apport des artefacts du pipeline ; elle mesure le comportement du moteur de recherche KB lui-même : type de recherche, seuil sémantique, nombre de documents sémantiques et nombre de documents full-text.

Stratégie	**Search**	**Thr.**	**Sem. docs**	**FT docs**	**QI**	**Lecture**
S0 Current	Hybrid	0.55	25	12	65	Équilibre précision/rappel
S1 Recall+	Hybrid	0.45	35	15	**66**	Meilleur score mesuré
S2 Precision+	Hybrid	0.65	18	8	63	Trop restrictif
S3 Semantic-heavy	Hybrid	0.55	35	6	51	Ancrage exact insuffisant
S4 Fulltext-heavy	Hybrid	0.55	15	20	47	Reformulations mal couvertes
S5 Semantic-only	Semantic	0.55	25	0	41	Ablation défavorable
S6 Fulltext-only	Full-text	–	0	20	N/A	Test en cours

Interprétation : S1 améliore légèrement le score global (+1 point vs S0) en augmen-

tant le rappel. Cependant, l'écart faible entre S0 et S1 montre que le gain de couverture est presque compensé par l'augmentation du bruit. Les configurations S3, S4 et S5 démontrent que le système a besoin d'un équilibre réel entre sémantique et full-text : trop de sémantique perd les noms exacts d'API, trop de full-text perd les reformulations utilisateur.

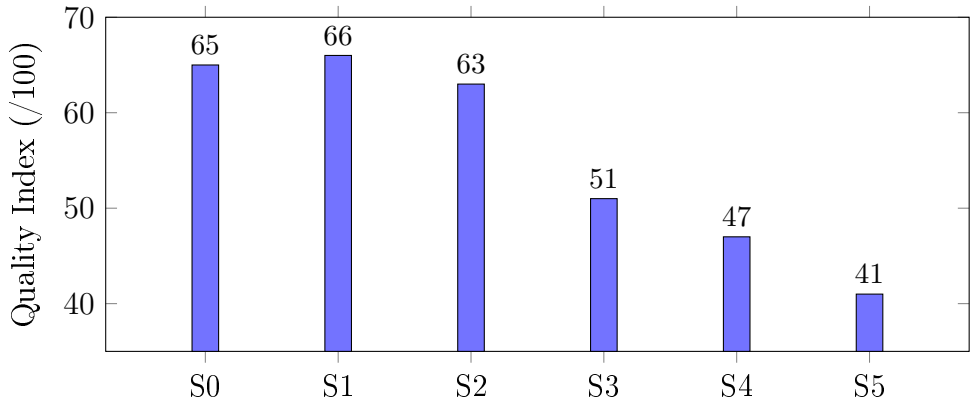


FIGURE 8.5 – Comparaison des stratégies Unstructured KB sur 96 questions multi-séries

8.6.3 Matrice Valeur × Effort par Étape

Étape	Impact QI	Effort dev.	ROI
Diagnostic Cards	+33 pts	Moyen (2 sprints)	**Excellent**
Hybrid Search	+4 pts	Faible (config)	**Excellent**
System Prompt	+3 pts	Faible (texte)	**Excellent**
Cleaning V3	+3 pts estimé	Moyen (1 sprint)	Bon
NovaX restrictif	Protège le routage KB-first	Faible (description)	**Excellent**
PDF descriptions	+1 pt estimé	Élevé (CV + Alfred)	Acceptable

TABLE 8.25 – ROI de chaque amélioration du pipeline sur le Quality Index

Chapitre 9

Automatisation et CI/CD

9.1 GitHub Actions — Mise à Jour Automatique

Le pipeline est automatisé via GitHub Actions avec un workflow bi-mensuel :



FIGURE 9.1 – Architecture CI/CD : GitHub Actions + self-hosted runner

9.1.1 Architecture 2 Jobs

1. **Job 1 — Pipeline** (cloud, ubuntu-latest) : Ingestion + Preprocessing + Export ST-ready

2. **Job 2 — Upload** (self-hosted, PC local) : Delete ancien contenu + Upload nouveau vers KB #793

9.1.2 Déclenchement

— Automatique : 1er et 15 de chaque mois à 03h UTC

— Manuel : via l'interface GitHub (workflow_dispatch) avec sélection de séries

9.1.3 Sécurité et Robustesse

— Gate de validation schéma (bloque l'upload si les JSON sont malformés)

— Delete + Re-add pour éviter les conflits de noms de fichiers

— Logs horodatés et artefacts archivés 30 jours

— `-continue-on-error` permet de compléter les séries restantes même si une échoue

Chapitre 10

Conclusion et Perspectives

10.1 Bilan du Projet

Ce projet de fin d'études, réalisé au sein de l'équipe Support & Maintenance de ST-Microelectronics Tunis, a permis de concevoir, implémenter et déployer un **pipeline de prétraitement intelligent** qui transforme les données brutes des dépôts GitHub STM32Cube en une base de connaissances structurée, alimentant un chatbot IA spécialisé en production sur la plateforme ST ChatGPT.

Les réalisations principales sont :

— Un **pipeline modulaire en 7 étapes** (Ingestion → Nettoyage V1+V3 → Enrichissement NLP → Similarité TF-IDF → Chunking → Validation → Delivery), entièrement rejouable par stage avec des artefacts JSON traçables

— La couverture de **13 séries MCU en production** (C0, F0, F1, F2, F3, F4, F7, G0, H5, H7, H7RS, U5, WL) totalisant 47 dépôts, plus **12 séries supplémentaires configurées** (G4, L0, L1, L4, L5, N6, U0, U3, WB, WB0, WBA, WL3)

— La production de **4 types d'artefacts complémentaires** : issues structurées (∼2 000), fichiers documentés (∼14 000), resolver cases (∼390) et diagnostic cards (∼157)

— Un **nettoyage intelligent V3 par type** avec réduction de 96% pour les headers CMSIS et 33% pour les Release Notes HTML

— L'**enrichissement IA multi-modal** via Alfred Vision : description automatique des captures d'écran d'issues (taux de succès >98% sur H7, F4, WL et F7), évaluation de cohérence sur 200 images d'issues (seulement 1.5% à risque), et 242 figures PDF détectées sur 19 séries avec une confiance contextuelle encore perfectible

— L'**automatisation CI/CD** via GitHub Actions et scripts PowerShell/Python

— Un **Quality Index de 94/100** sur la série pilote H7 (avec Diagnostic Cards) et un **gain de +39%** vs Alfred générique sur les questions techniques spécifiques

10.2 Valeur Industrielle

La valeur industrielle du projet se mesure concrètement :

Scénario	Sans KB	Avec KB
Trouver un bug HAL connu	30–60 min	**2 min**
Identifier la version du fix	Lecture du changelog	**Réponse directe**
Tracer Issue → PR → Commit	Navigation manuelle	**Resolver Case**
Comprendre une figure PDF	Ouvrir le PDF, chercher la page	**Description textuelle**
Ajouter une nouvelle série MCU	Développement custom	**3 étapes documentées**

10.3 Leçons Apprises

Ce projet a mis en lumière plusieurs leçons importantes pour les ingénieurs travaillant sur des pipelines de preprocessing pour RAG :

1. **L'intégrité JSON est fragile** : l'injection de texte brut dans des chaînes JSON peut corrompre les fichiers si les caractères spéciaux ne sont pas correctement échappés. Nous avons développé des scripts de réparation (`fix_corrupted_json_files.py`) après avoir rencontré ce problème en production.
2. **La description du tool contrôle le routage du LLM** : dans une architecture multi-tools, une description trop permissive du tool fallback (Alfred) dégrade le Quality Index de 10 points en court-circuitant la KB. La rédaction de cette description est un paramètre de configuration critique.
3. **Le passage à l'échelle dilue le contexte** : passer de 1 à 13 séries a fait chuter le QI de 94 à 54, nécessitant un travail d'optimisation des métadonnées et du filtrage par série.
4. **Les heuristiques domain-specific surpassent le ML générique** : dans un contexte où les données d'entraînement supervisées sont absentes, les règles regex spécifiques au domaine STM32 fournissent une classification fiable et explicable.

10.4 Perspectives

Plusieurs axes d'amélioration et d'extension sont envisagés :

1. **Évaluation A/B V2 vs V3** : comparer le Quality Index avec les datasources V3 actives vs V2 pour quantifier l'impact exact du nettoyage intelligent sur la qualité des réponses
2. **Extension aux séries configurées** : déployer les 12 séries restantes (G4, L0, L1, L4, L5, N6, U0, U3, WB, WB0, WBA, WL3) avec création des datasources KB
3. **Enrichissement sémantique** : remplacer TF-IDF par des embeddings pré-calculés pour améliorer la couverture de similarité (actuellement 22% sur H7)
4. **Métriques RAG avancées** : intégrer des métriques automatisées (Faithfulness, Relevance) pour une évaluation continue sans intervention humaine
5. **Graphe de connaissances** : explorer la construction d'un graphe reliant issues, composants, boards et versions pour faciliter le raisonnement multi-hop
6. **Filtrage par série dans le retrieval** : instruire la persona pour inclure systématiquement le nom de la série dans ses queries KB, réduisant la dilution multi-séries
7. **Multi-langue** : support des issues en chinois et japonais (communauté asiatique significative)
8. **Nettoyage V4 adaptatif** : utiliser un classifieur supervisé entraîné sur les résultats V3 pour identifier automatiquement les sections utiles des fichiers source

Références

Bibliographie

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. et Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474. — Article fondateur du paradigme RAG que nous appliquons : recherche documentaire + génération par LLM. Le principe de base de notre architecture KB + ST ChatGPT.

[2] Salton, G. et Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, vol. 24, no. 5, pp. 513–523. — Référence fondamentale sur TF-IDF, la méthode de pondération de termes que nous utilisons dans notre module de calcul de similarité inter-issues (`compute_issue_similarity_v2.py`).

[3] OpenAI (2023). *GPT-4 Technical Report*. arXiv :2303.08774. — Le modèle GPT (dans sa version Azure OpenAI) est le LLM utilisé par la plateforme ST ChatGPT pour la génération de réponses, et par Alfred Vision pour la description d'images.

[4] STMicroelectronics (2024). *STM32Cube MCU & MPU Packages — Open Source Firmware*. https://github.com/STMicroelectronics. — Les dépôts GitHub de STMicroelectronics : source principale de données de notre pipeline (issues, fichiers, PRs, commits).

[5] STMicroelectronics (2024). *Annual Report 2023 — Empowering the Smart Edge*. Genève, Suisse. — Rapport annuel fournissant les données sur le chiffre d'affaires (17,3 milliards USD), les effectifs (50 000+ collaborateurs) et la répartition des marchés.

[6] ARM Limited (2024). *ARM Cortex-M Processor Family*. https://developer.arm.com/Processors/Cortex-M. — Documentation de référence sur les cœurs ARM Cortex-M (M0, M0+, M3, M4, M7, M33, M55) utilisés dans les microcontrôleurs STM32.

[7] Schwaber, K. et Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org. — Guide officiel de la méthodologie Scrum, base de notre approche de pilotage par sprints itératifs de 2 semaines.

[8] Wright, A., Andrews, H. et Hutton, B. (2022). *JSON Schema : A Media Type for Describing JSON Documents*. Internet Engineering Task Force (IETF), draft-bhutton-json-schema-01. — Standard JSON Schema utilisé pour nos gates de validation (`validate_schemas.py`) qui bloquent l'upload en cas de non-conformité.

[9] ISO/IEC (2008). *ISO/IEC 25012 :2008 — Systems and software engineering – SQuaRE – Data quality model*. International Organization for Standardization. — Référence du modèle de qualité des données (complétude, cohérence) utilisé pour justifier les métriques de couverture/complétude par ratios.

[10] ISO/IEC (2015). *ISO/IEC 25024 :2015 — Systems and software engineering – SQuaRE – Measurement of data quality*. International Organization for Standardization. — Référence sur la mesure quantitative de la qualité des données, alignée avec nos indicateurs par stage.

[11] ISO/IEC (2011). *ISO/IEC 25010 :2011 — Systems and software engineering – SQuaRE – System and software quality models*. International Organization for Standardization. — Référence du modèle de qualité (dont la fiabilité), utilisée pour cadrer les indicateurs de fiabilité opérationnelle Alfred.

[12] Manning, C. D., Raghavan, P. et Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. — Référence de base sur le compromis précision/rappel et le réglage des seuils de récupération d'information.

[13] Fawcett, T. (2006). *An introduction to ROC analysis*. Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874. — Référence méthodologique pour la sélection et la justification de seuils décisionnels.

[14] Microsoft (2024). *Azure OpenAI Service Documentation*. https://learn.microsoft.com/azure/ai-services/openai/. — Plateforme cloud sur laquelle est déployé ST ChatGPT (Azure OpenAI) et Alfred Vision.

[15] Py-PDF Contributors (2024). *pypdf — A pure-Python PDF library*. https://pypdf.readthedocs.io/. — Bibliothèque Python utilisée dans notre pipeline pour l'extraction du texte et la détection de figures dans les Release Notes PDF.

Annexes

A. Liste des Diagrammes

Diagrammes de présentation :

1. `01_pipeline_overview.drawio` — Vue d'ensemble du pipeline (Figure 5.1)
2. `02_delivery_kb_upload.drawio` — Architecture delivery + upload (Figure 5.4)
3. `03_data_flow_source_to_answer.drawio` — Flux source-to-answer (Figure 5.3)
4. `04_series_coverage_scale.drawio` — Couverture multi-séries
5. `05_alfred_vision_integration.drawio` — Intégration Alfred (Figure 6.6)
6. `06_cicd_automation.drawio` — CI/CD GitHub Actions (Figure 9.1)

Diagrammes techniques :

1. `01_ingestion_detail.drawio` — Détail ingestion (Figure 6.1)
2. `02_enrichment_similarity_detail.drawio` — Enrichissement + Similarité (Figure 6.3)
3. `03_delivery_resolver_diagnostic.drawio` — Resolver + Diagnostic (Figure 6.4)
4. `04_chunking_v2_strategy.drawio` — Stratégie de chunking V2
5. `05_evaluation_validation.drawio` — Évaluation et validation
6. `06_automation_ci_upload.drawio` — Automation CI upload
7. `07_upload_auth_flow.drawio` — Flux d'authentification upload

Note : Pour intégrer les diagrammes dans le PDF, exporter les fichiers .drawio en PNG (File → Export as → PNG, 150 DPI) dans le dossier `docs/diagrams/{presentation,technical}/expo`

B. Commandes d'Exécution

Listing 10.1 – Pipeline complet

```
# Activer l'environnement
.venv\Scripts\activate
```

```
# Pipeline complet avec delivery (serie specifique)
$env:STM32CUBE_CONFIG = "shared/config/config_f4.json"
python -m pipeline.run_full_workflow --export-st-ready --continue-
    on-error

# Pipeline sans chunking (ST platform gere le chunking)
python -m pipeline.run_full_workflow --export-st-ready --skip-
    chunking

# Pipeline sans ingestion (reutilise les donnees existantes)
python -m pipeline.run_full_workflow --export-st-ready --skip-
    ingestion
```

Listing 10.2 – Enrichissement Alfred

```
# Alfred Vision - images d'issues
python pipeline_Automation/alfred/enrich_json_images_with_alfred.py
    \
  --input <fichier_issues_with_images.json> --inplace

# Alfred Vision - figures PDF
python pipeline_Automation/alfred/enrich_pdf_figures_with_alfred.py

# Patch descriptions PDF dans les delivery files
python pipeline_Automation/patch_pdf_descriptions_in_delivery.py

# Fix JSON corrompus (control characters)
python pipeline_Automation/fix_corrupted_json_files.py
python pipeline_Automation/fix_corrupted_json_deep.py
```

Listing 10.3 – Upload vers KB #793

```
# Upload issues Alfred-enrichies (13 series)
python pipeline_Automation/upload/upload_alfred_issues_all_series.
    py

# Upload files PDF-enrichies (13 series)
python pipeline_Automation/upload/
    upload_pdf_enriched_files_all_series.py

# Upload drivers sous-repos (8 nouvelles series)
```

```
.\pipeline_Automation\upload\Upload_All_Drivers.ps1

# Upload manuel (un fichier)
python pipeline_Automation/upload/Add_Data_Source_Files.py \
  --kb 793 --operation add --datasource-id <ID> \
  --remote-user "abir.gharbi@st.com" \
  --processor JSON --processor-params "base64:<b64>" \
  --json-split-size 1000 --files <fichier.json>
```

Listing 10.4 – Validation et evaluation

```
# Validation des schemas
python -m pipeline.evaluation.validate_schemas

# Stats issues
python -m pipeline.evaluation.test_stats_issues_v2

# Stats files
python -m pipeline.evaluation.test_stats_files_v2

# Recherche TF-IDF (test local)
python -m pipeline.evaluation.test_tfidf_search_issues_v2
```

Listing 10.5 – Ajout d'une nouvelle serie (ex : G4)

```
# 1. Config deja creee : shared/config/config_g4.json
# 2. Lancer le pipeline
$env:STM32CUBE_CONFIG = "shared/config/config_g4.json"
python -m pipeline.run_full_workflow --export-st-ready \
  --skip-chunking --continue-on-error
# 3. Creer datasources sur KB #793 (interface web)
# 4. Mettre a jour config_all_series.json avec les IDs
# 5. Uploader
python pipeline_Automation/upload/upload_alfred_issues_all_series.
   py
python pipeline_Automation/upload/
   upload_pdf_enriched_files_all_series.py
```

C. Structure JSON — Resolver Case (Extrait)

```
{
  "resolver_cases": [
    {
      "issue_number": 42,
      "repo": "stm32cubef4",
      "label": "SPI DMA TransmitReceive fails...",
      "resolution_status": "merged_fix",
      "link_confidence": "high",
      "primary_pr_number": 87,
      "primary_changed_file": "stm32f4xx_hal_spi.c",
      "resolver_card_text": "Problem: ...",
      "impacted_modules": ["SPI", "DMA"],
      "impacted_boards": ["NUCLEO-F446RE"]
    }
  ]
}
```